



Online Learning from data streams via decentralized and asynchronous SGD

Mauro Dalle Lucca Tosi ^{a,*,}, Martin Theobald ^b

^a Human Centered AI, Data & Software Unit, Luxembourg Institute of Science and Technology, Esch-sur-Alzette, Luxembourg

^b Department of Computer Science, University of Luxembourg, Esch-sur-Alzette, Luxembourg

ARTICLE INFO

Keywords:

Online Learning
Data streams
Concept drifts
Asynchronous & decentralized SGD
Artificial neural networks

ABSTRACT

Online Learning (OL) is a sub-field of Machine Learning (ML) which focuses on solving time-sensitive problems through iterative learning from data streams. This emerging field is characterized by the challenge of *concept drifts*, where the underlying distribution of the incoming data values evolves over time. Traditional OL algorithms, while efficient and less resource-intensive than conventional ML methods, often fall short in solving non-linear, high-dimensional problems. This prevalent gap has recently led to the integration of Artificial Neural Networks (ANN) into OL settings. These models support real-time inference. However, because they rely on offline training, their performance often degrades during or shortly after concept drifts. In this paper, we extend TensAIR, an online stream-processing engine that we specifically designed for the distributed training of ANN models. Our extensions allow TensAIR to automatically identify concept drifts using the OPTWIN drift detector algorithm, triggering the retraining of the ANN models as soon as drifts are detected. Additionally, we propose a novel *decentralized and asynchronous stochastic gradient descent* (DASGD) algorithm, which is central to TensAIR's performance improvements over existing methods, and we formally prove its convergence under the specified conditions. We assessed TensAIR both in single-server and HPC settings, evaluating its distributed training performance over various multi-CPU and multi-GPU scenarios. As result, we show TensAIR to converge within the best known theoretical bounds while achieving up to 78× higher sustainable throughput than state-of-the-art baselines. Based on our results, we expect to inspire further research and applications exploiting the distributed training of ANN models in HPC platforms for a wide range of OL settings.

1. Introduction

Online Learning (OL) recently emerged as a noteworthy sub-field within Machine Learning (ML) with increasing attention from both academia and industry. This specialized area of ML focuses on developing solutions that address problems by iteratively learning from individual data samples in a sequential manner [1]. The growing volume of generated data [2], as well as advancements in technology over recent decades [3], have propelled OL to the forefront of research, particularly in scenarios where real-time responses are imperative and based on the latest available data samples. Notable real-world applications of OL include Fraud Detection [4], Dynamic Pricing [5], Stock Market Prediction [6], among others [7].

Unlike traditional ML methods, OL solutions operate without relying on pre-defined datasets for training. Instead, they adapt to the dynamic nature of data streams, where individual data samples are continuously produced over time [1]. Formally, a data stream Ω is characterized by an ordered sequence of potentially infinite events e_i with corresponding timestamps ω_i expressed as $\Omega = (e_1, \omega_1), \dots, (e_\infty, \omega_\infty)$, where ω_i denotes

the processing time i at which the event e_i is incorporated into the system.

Due to the distinctive nature of training data sources, we underscore two key differences between standard ML and OL settings. Firstly, in standard ML, it is customary to iterate over the same data sample multiple times during training, known as “epochs” [8]. Conversely, in OL, a data sample is ideally processed only once [1], due to the unbounded nature of data streams.

Secondly, OL settings often face *concept drifts*, defined as unforeseeable changes in the statistical properties of incoming data over time [9]. While traditional ML assumes fixed data distributions, OL requires models to update incrementally as the underlying distribution evolves, ideally in real-time. Thus, pre-trained models often fail to maintain performance after a concept drift, as they lack the capacity for online adaptation.

To cope with such changes, OL models must be retrained or adapted promptly. Conventional OL methods often rely on lightweight models (e.g., Naïve Bayes [10], Random Forests [11]) that are well-suited for fast updates but insufficient for complex data such as audio, video,

* Corresponding author.

E-mail addresses: mauro.dalle-lucca-tosi@list.lu (M.D.L. Tosi), martin.theobald@uni.lu (M. Theobald).

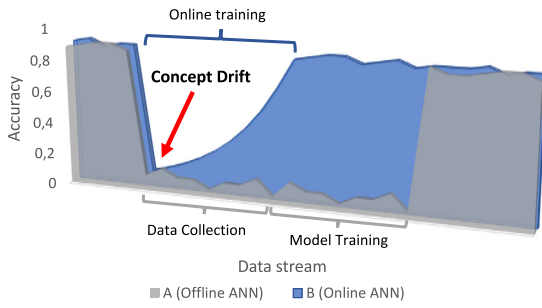


Fig. 1. Illustration of the impact of delayed adaptation on the quality of predictions in an ANN under concept drifts.

or high-dimensional inputs [12]. Addressing these challenges with Artificial Neural Networks (ANNs) is still a largely open problem, due to their high computational cost and data requirements. Consequently, most ANN-based systems are trained offline and used only for inference in OL scenarios [13,14], which causes delayed adaptation after concept drifts and yields suboptimal predictions (Fig. 1).

Despite the need, current research lacks efficient approaches for online training of ANNs. In particular, existing approaches rarely support fully decentralized and asynchronous optimization, which is key for exploiting the resources of High-Performance Computing (HPC) systems without centralized bottlenecks or synchronization delays. While asynchronous stochastic gradient descent (ASGD) and decentralized SGD (DSGD) have each been studied individually, combining both properties in a single algorithm with theoretical convergence guarantees remains rare and technically challenging—especially in non-convex settings relevant to ANNs.

To address these challenges, we extend our previously proposed TensAIR system [15], enabling efficient online training of ANNs on streaming data by supporting both decentralized and asynchronous computation. Additionally, we incorporate active concept drift detection into the training pipeline using the OPTWIN algorithm [16], thereby minimizing unnecessary retraining and further improving adaptation efficiency.

Contributions. This article's main contributions are:

- We extend TensAIR to enable decentralized and asynchronous training of ANN models in OL settings, actively adapting to concept drifts detected by the OPTWIN algorithm.
- We formally propose DASGD, a novel decentralized and asynchronous stochastic gradient descent algorithm, and prove its convergence for non-convex, homogeneous, and L-smooth objective functions—both with and without bounded gradients.
- We implement a high-level Python interface to deploy TensAIR in real HPC environments, and empirically demonstrate its scalability and performance improvements over existing OL stream-processing systems, achieving up to 78× higher throughput.

The remaining of this article is organized as follows. Section 2 gives an overview of the background areas and related works. Section 3 presents the TensAIR system. Section 4 describes the OPTWIN concept drift detector integrated into TensAIR. Section 5 defines DASGD and presents its convergence proof. Section 6 describes the experimental evaluation. Section 7 presents the results of the experimental evaluation. Section 8 discusses the results. Section 9 concludes the article.

2. Background & related works

2.1. Stream processing engines

Stream Processing Engines were developed to address the real-time processing of samples from data streams. In order to reduce the

time taken to process each data sample, stream processing engines usually allow its execution to be distributed across multiple nodes. By doing so, they end up increasing their sustainable throughput—the number of data samples they can process per second. Among the most popular stream processing engines are Apache Kafka [17] and Apache Flink [18], widely utilized by researchers and practitioners in processing data streams for OL problems. Despite their high performance and fault tolerance, they still suffer from bottlenecks due to their centralized architecture [19]. Different from the most popular engines mentioned above, AIR [19] is a research-oriented stream processing engine which uniquely adopts a fully asynchronous and decentralized architecture. Notably, AIR demonstrated superior performance, sustaining up to 15 times more throughput than traditional engines like Kafka and Flink [20] in regular stream-processing settings.

2.2. Practical usage of ANNs in online learning

Both Apache Kafka and Flink enable ANNs to process data from streams. Kafka supports TensorFlow [14] but does not support its usage in a distributed manner, which limits its applicability in practice. On the other hand, Apache Flink implementation has a third-party library [13] that allows the distributed usage of the TensorFlow library. Still, both implementations were optimized to deploy pre-trained models and use the data samples from the data stream simply for inferences or predictions. When stream data is used for training, both suffer from high overhead due to large data transfers necessary to train ANN models (cf. Section 6.3), limiting their sustainable throughput.

Different from the solutions presented above, we designed TensAIR [15] from scratch for training ANNs via a form of *decentralized and asynchronous SGD* (DASGD) (cf. Section 5). To guarantee high sustainable throughput and reduced overhead, TensAIR is based on the AIR stream processing engine and TensorFlow (both openly available as native C/C++ code).

In our first paper presenting TensAIR [15], we introduced the TensAIR framework to allow the ANN models to adapt to concept drifts in a passive manner (training with every new data sample that arrived from the data stream). However, such approach spends unnecessary computational time training ANN models that have already converged and did not suffer from new concept drifts since. Considering this, in this paper we extend TensAIR to perform active concept drift adaptation (only retraining the ANN models when a new concept drift is detected, and pausing the training after the model converges). To do so, we rely on OPTWIN [16], an error-based concept drift detector algorithm we designed for identifying concept drifts from the loss of ANNs. Furthermore, in [15], we presented a first version of the DASGD algorithm but did not provide its convergence proof. In this paper, we present an improved version of the DASGD algorithm, provide its convergence bounds in Section 5, and formal proof in Appendix B.

2.3. Theoretical usage of ANNs in online learning

Addressing the challenge of training ANN models in an online setting, Online Sequential Extreme Learning Machine (OS-ELM) [21] applies Extreme Learning Machines (ELM) models in the OL settings. ELMs are Feedforward Neural Networks with randomly assigned weights that only train their last layer via a linear solution [22]. This characteristic allows ELMs to be fast to train compared to other ANN architectures while still producing satisfactory results. However, ELMs are known to face performance issues when dealing with complex problems as exemplified in [23].

In contrast, [7] presented a use-case involving fake-news detection trained under an OL setting. They explored the online adaptation of an ANN model to enhance fake-news detection over a data stream continually experiencing concept drifts. Their study exemplifies a use-case that may not be solvable with classical OL solutions and could yield unsatisfactory results if not trained online, depending on the

frequency and magnitude of the concept drifts. The primary focus of their research is on addressing the continual learning problem, thus adapting to concept drifts without forgetting previous knowledge [24]. However, they did not delve into the practical challenges of implementing their use-case in a real-world setting, which we believe might be unfeasible with current solutions. Thus, showcasing how TensAIR could be employed to solve such problems, that are not possible with current technologies.

2.4. Distributed training of ANNs

ANNs are mostly trained via variations of the common Stochastic Gradient Descent (SGD) algorithm (such as mini-batch SGD [25], ADAM [26], AdaGrad [27], etc.). Despite its versatility and excellent performance, SGD demands a substantial amount of iterations to converge when solving complex problems over a large number of parameters. Consequently, training a large model with SGD may be a time-consuming task [28]. To mitigate this issue, it is common to distribute the computations performed by the SGD optimizer across multiple CPUs, GPUs, or even across multiple compute nodes. Various settings used to distribute the SGD optimizations are described in the literature [29,30]; in this paper, we will focus on *data parallelism*, which is the most common form of distributed training for SGD-based optimizers [30].

Decentralized SGD. The idea that decentralized SGD (DSGD) can outperform centralized SGD has been shown in [31]. However, depending on the topology of the underlying network, the communication cost when not relying on a centralized parameter server can increase quadratically with the number of distributed worker nodes n [30]. One approach to lower the communication cost is to use the so-called “gossip algorithms” [31,32], which allow workers to exchange and aggregate their gradients only with their immediate neighbors (usually 2 to 3), instead of all worker nodes. Then, those neighbors disseminate the messages received across the network iteratively, and after approximately $1.639 \log_2(n)$ communication steps all workers will receive the initial message [33]. However, networks with more than 32 workers may still suffer in terms of convergence and performance due to their relatively high delay in the exchange of messages [33].

Asynchronous SGD. Introduced already decades ago, the study of asynchronous SGD (ASGD) has gained an increasing amount of attention again recently. At first, works such as Hogwild! [34] focused on proving the convergence of ASGD under the assumption of the sparseness of the optimized models. Recently, less restricted convergence proofs were developed, which also contemplate dense models as in [35–41]. A main aspect to facilitate the proof of convergence used by [36] is to scale the gradients based on their delay, such that delayed gradients have less impact when updating the model. Another technique seen in [37] is to simply discard gradients which are too delayed. Both techniques described above are easy to implement in a centralized setting, in which a parameter server can adapt its learning rate based on the delay of the gradient that it applies. However, this adaptation is not straightforward in a decentralized setting, in which there is no single server to decide how delayed the gradients are. Furthermore, part of these ASGD convergence proofs do not cover the optimization of non-convex functions [38,39]. Thus, only very few convergence proofs provided in the literature, such as [35,40,41], cover ASGD on non-convex objective functions while using learning rates which are invariant to the gradients’ delays. We highlight here the recent proof by Koloskova, Stich and Jaggi [35] which provides the currently best convergence rate for ASGD under relatively mild assumptions, while considering a fixed learning rate and non-convex functions. Specifically, they were able to prove that ASGD convergence does not depend on the maximum delay of gradients when those gradients are bounded.

Decentralized & Asynchronous SGD. To mitigate the bottleneck of using a centralized parameter server and to avoid idle times from

synchronizing distributed models, some authors focused their research on decentralized and asynchronous SGD [42–49]. However, most of those studies make strong assumptions as in [42–46], which either do not prove convergence for non-convex functions, or they prove the convergence of their models only when the number of iterations goes to infinity [42,46,47]. Contrarily, [48] proved the convergence rate of decentralized and asynchronous SGD for non-convex functions by relying on specific network topologies and atomic communication steps among nodes, which requires a partial synchronization between any two nodes during the optimization process, thus hindering the convergence time in practice [50]. Differently, [49] does not assume atomic communication steps but blocks its workers to ensure that a maximum delay τ is maintained during the optimization process.

Recently, [51] reached a similar convergence rate to the one we present in this paper by also using a wait-free approach that eliminates the partial synchronization and special network architecture requirements. Their optimization process relies on a novel client-communication matrix that governs the models’ averaging. As opposed to this approach, we freely exchange gradients among models and introduce the new *staleness* metric to measure the difference between those models based on the sets of gradients they exchanged. At last, both our approach and the one presented in [51] were able to eliminate the dependency on the slowest model (the “maximum delay”) in the network from their convergence rate proofs. We eliminate this dependency by assuming that gradients are bounded, while they eliminate this dependency by defining a minimum amount of iterations until convergence and by choosing their learning rate accordingly.

In this article, we build upon our previous work on the TensAIR architecture [15] by introducing several key enhancements. First, we extend the architecture to support active concept drift detection. Second, we implement OPTWIN, an active and adaptive mechanism specifically designed to detect and react to concept drift in ANNs. Third, we improve the DASGD optimization algorithm by refining its convergence analysis and establishing formal bounds. The following three sections describe each of these contributions in detail.

3. TensAIR architecture

To effectively illustrate TensAIR’s operation, we represent its dataflow as a directed graph, where vertices correspond to system operators and edges represent the communication pathways between them. In our OL setting, TensAIR’s configuration relies on two primary operators: the “Mini Batch Generator”, which compiles mini-batches from the incoming data stream, and the “Model”, which manages distributed instances of artificial neural networks (ANNs). This setup enables a form of passive drift adaptation, where model instances are continuously trained on the most recent data samples without explicitly detecting concept drifts [15]. In this article, we enhance this framework by introducing an active drift adaptation mechanism (see Fig. 2), in which models are retrained only when a concept drift is detected. This strategy improves computational efficiency by avoiding unnecessary retraining—either when no drift occurs or when a detected drift does not negatively impact a model instance (i.e., its prediction accuracy remains unchanged).

In Fig. 2, we illustrate our enhanced TensAIR dataflow. This setup involves three key components: the “Mini Batch Generator”, the “Model”, and the “Drift Detector”. Each plays a crucial role in the system’s operation. Specifically, the “Mini Batch Generator” acts as the entry point to this workflow. It takes data samples from the data stream and compiles them into mini-batches. These mini-batches are then used for either prediction or training purposes, depending on the model’s current state. Next in sequence is the “Model” operator. This component is responsible for initializing and managing n instances of ANNs, distributed across n ranks. These “Model” instances receive mini-batches from the “Mini Batch Generator”. When the models are in a stable state, having converged and not having encountered any concept

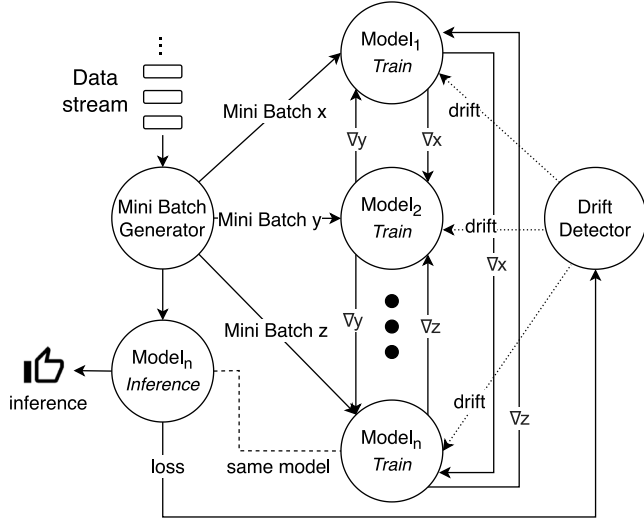


Fig. 2. TensAIR dataflow with active drift adaptation.

drift since their last training iteration, they use the incoming mini-batches for making predictions. The outcomes of these predictions, particularly the losses incurred, are then forwarded to the “Drift Detector”.

However, if the “Model” instances are still adapting to a recent concept drift — meaning they have not yet converged — the mini-batches are instead used to continue training. During training sessions, the “Model” instances process the mini-batches, thereby calculating and applying the necessary SGD updates. These updates are broadcast across all model instances to ensure uniform learning and adaptation. This training continues until all “Model” instances have converged (cf. Section 5), at which point it transitions back to using the mini-batches for inference until a concept drift is flagged by the “Drift Detector”, as it is detailed in Algorithm 1.

Algorithm 1 TensAIR Algorithm [15,52].

```

1: training = True
2: bf = broadcast_frequency
3: Δ = ∅
4: delta_count = 0
5: procedure STREAMPROCESS(msg)
6:   if msg from Mini Batch Generator then
7:     if training then
8:       gradient = calculateGradient(msg)
9:       apply_gradient_locally(gradient)
10:      Δ = Δ + gradient
11:      delta_count = delta_count + 1
12:      if delta_count = bf then
13:        broadcast(Δ)
14:        delta_count = 0
15:        Δ = ∅
16:      converged = checkConvergence()
17:      if converged then
18:        training = False
19:      else
20:        prediction, loss = predict(msg)
21:        send_to_drift_detector(loss)
22:      else if msg from Model then
23:        apply_gradient_locally(msg)
24:        converged = checkConvergence()
25:        if converged then
26:          training = False
27:        else if msg from Drift Detector then
28:          training = True

```

In Algorithm 1, we describe the operation of the “Model” operator for each new message received. First, the necessary variables are initialized. Specifically, *training* flags if the model is in training or prediction

mode; *bf* is a hyper-parameter that decides how many gradients a model must calculate before broadcasting them to other models; Δ aggregates gradients not yet broadcast; *delta_count* saves the number of gradients already aggregated in Δ . After the variables are initialized, in Lines 6, 22 & 27 the algorithm checks which operator sent the message. If the message was sent from the “Mini Batch Generator”, it checks whether to use the received mini-batch for training or prediction (Line 7). If in training mode, it follows the training process of the DASG algorithm, as explained in details in Section 5, by calculating and applying the gradient correspondent to the received mini-batch (Lines 8 & 9) and, if *bf* gradients have already been aggregated, broadcasting the gradient to the other distributed models (Line 13). Then, at Line 16, it checks if the model has converged.¹ If it has converged, it deactivates the training mode at Line 18. Differently, if a message is received from the “Mini Batch Generator” and the “Model” is not in training mode, the mini-batch is used for prediction (Line 20), having its loss sent to the “Drift Detector” (Line 21). On the other hand, if the message first received was sent by another “Model” instance (i.e., from a different rank), the message is a gradient that has been broadcast and thus must be locally applied first. In this case, the gradient is applied (Line 23), convergence is checked (Line 24), and training model is deactivated if the model has already converged (Line 26). At last, if the model receives a message from the “Drift Detector”, it indicates that a concept drift occurred and the model must change to training mode (Line 28).

The final and novel piece in this process is the “Drift Detector”. This operator is critical for identifying changes in the data stream that might affect model performance. It examines the prediction losses received from the “Model” instances and determines, via OPTWIN [16] (cf. next section) if a concept drift occurred or not. If OPTWIN identified a concept drift, the “Drift Detector” signals the “Model” to shift its focus back to training mode, mitigating the concept drift phenomena.

4. Optimal window (OPTWIN) drift detector

OPTWIN [16] is the “OPTimal WINDOW” drift detector algorithm that we developed to identify concept drifts specifically from the losses of ANNs. It stores the error rates from a possibly unbounded data stream $X = r_1, r_2, \dots, r_i, \dots$ produced by machine learning algorithms into a sliding window $W \subset X$. It then computes the *optimal splitting point* r_i of W , thereby dividing it into two subwindows W_{hist} and W_{new} , which each contain the historical and newest errors from W , respectively. We refer to OPTWIN’s split as “optimal” because it minimizes the size of W_{new} (requiring the fewest new elements) while enabling the detection of statistically meaningful changes with a *robustness* of at least ρ , a hyper-parameter that quantifies the minimum detectable concept drift based on the data’s standard deviation, which we explain in details later in this section. Consequently, once the sub-windows are identified, OPTWIN applies both the *t*- and the *f*-tests to detect any potential concept drift based on the mean and standard deviation of the errors produced by the underlying ML algorithm. Algorithm 2 outlines the OPTWIN algorithm.

In Algorithm 2, OPTWIN takes as input the confidence level δ , the robustness parameter ρ , and w_{max} , the maximum allowable size for the sliding window W . Additionally, it initializes β to a small value to avoid divisions by zero along with the sliding window W and w_{min} , the minimum permissible size of W .

Each time a new error r_i is computed — in our case, the loss of an ANN — the OPTWIN procedure is initiated. Between Lines 2 and 6, OPTWIN ensures that the sliding window’s size remains above w_{min}

¹ A simple and practical metric we adopt to check if a model has already converged is to check if its loss values have decreased less than a pre-defined threshold over the last z training samples. Still, more elaborate metrics and techniques to identify convergence of the models and their hyper-parameters can be used, but their study is beyond the scope of this paper.

Algorithm 2 OPTWIN Algorithm [16].

Input parameters: • δ – confidence level • ρ – robustness • w_{max} – max window size	Global variables: $W = \langle \rangle$ – sliding window $w_{min} = 30$ – min window size $\beta = 1e^{-5}$ – avoids division by 0	
---	--	--

```

1: procedure ADD_ELEMENT( $r_i$ )
2:    $W \leftarrow W \cup r_i$ 
3:   if  $|W| < w_{min}$  then
4:     return False
5:   else if  $|W| \geq w_{max}$  then
6:      $W \leftarrow W - W_0$ 
7:    $v \leftarrow \text{OPTIMAL\_CUT}(|W|, \rho, \delta^{\frac{1}{4}})$ 
8:    $v_{split} \leftarrow \lfloor v |W| \rfloor$ 
9:    $W_{hist} \leftarrow W_{0:v_{split}}$ 
10:   $W_{new} \leftarrow W_{v_{split}:|W|-1}$ 
11:  if  $\frac{(\sigma_{W_{new}} + \beta)^2}{(\sigma_{W_{hist}} + \beta)^2} > f\_ppf(\delta^{\frac{1}{4}}, v |W| - 1, (1-v) |W| - 1)$  then
12:    reset()
13:    return True
14:  else if  $t\_value(W_{hist}, W_{new}) > t\_ppf(\delta^{\frac{1}{4}}, df)$  then
15:    reset()
16:    return True

```

and below w_{max} . At Line 7, it calculates the largest value of v , the optimal splitting point of $|W|$ that satisfies Eq. (1), given δ , ρ , and $|W|$. Subsequently, between Lines 8 and 10, OPTWIN divides W into W_{hist} and W_{new} based on v . Finally, in Lines 11 to 14, OPTWIN performs both the t - and f -tests (in any order), alerting the user if W_{hist} and W_{new} differ statistically in either test, thereby indicating a concept drift.

4.1. Optimal splitting point

The core component of OPTWIN is the computation of the optimal splitting point v based on the robustness parameter ρ as defined in Eq. (1).

$$\rho = t_ppf(\delta', df) \sqrt{\frac{1}{v|W|} + \frac{f_ppf(\delta', v|W|-1, (1-v)|W|-1)}{(1-v)|W|}} \quad (1)$$

The following assumptions underlie this calculation.

Assumptions:

- The first w_{min} losses from the ANN, used to initialize OPTWIN, are assumed to have no concept drift.
- A concept drift is defined as a statistically significant variation in the mean or standard deviation between any two sub-windows of W .
- The t_value (obtained from the unequal-variance t -test [53]) computed between any two sub-windows of W follows a t -distribution.
- The f_value (obtained from the f -test [54]) computed between any two sub-windows of W follows an f -distribution.

The first assumption enables the initialization of OPTWIN without being negatively affected by outliers. The second assumption defines a concept drift as a statistically significant change not only in the mean but also in the standard deviation between the sub-windows of W , thereby broadening the notion of drift compared to similar detection algorithms such as [55]. The final two assumptions are necessary to ensure the validity of the t - and f -tests. These assumptions do not constrain the practical implementation of the algorithm, as standard deviations are known to follow the f -distribution [56], and it is considered good practice to design machine learning models such that their errors are normally distributed [57]. Even if this assumption does not hold, a “false positive” concept drift detection would merely trigger model retraining, which could in turn help mitigate the impact of skewed data points.

Eq. (1) was derived by integrating the commonly used t - and f -tests, as detailed in Appendix A. In this equation, f_ppf and t_ppf denote the Probability Point Functions (PPF) of the f - and t -distributions, respectively. The user-specified confidence level is adjusted to $\delta' = \delta^{\frac{1}{4}}$ because this confidence is employed four times throughout the algorithm. The symbol df represents the degrees of freedom used in the calculation of t_ppf . Finally, ρ represents the ratio by which $\mu_{w_{new}}$, the average of the values in W_{new} , must differ from $\mu_{w_{hist}}$, the average of the values in W_{hist} , for the change to be considered a concept drift. Thus, ρ quantifies the minimum detectable concept drift based on σ_{hist} , the data’s historical standard deviation, detecting drifts when $\rho \sigma_{hist} \geq |\mu_{hist} - \mu_{new}|$.

Interestingly, aside from $|W|$, all variables required to compute v in Eq. (1) are specified before executing Algorithm 2. Given that $w_{min} \leq |W| \leq w_{max}$, it is therefore feasible to pre-calculate all optimal splitting points for W based on the anticipated sizes of the sliding window.

Additionally, we note that if $|W|$ is too small — specifically, below w_{proof} — it may not be possible to statistically guarantee the detection of small concept drifts. In such cases, we set $v = 0.5$, splitting W exactly on the middle until the sliding window grows sufficiently to detect drifts with a robustness of at least ρ . In practice, the definition of ρ is shown in [16] to be simple, generally producing good results with $\rho = 0.5$ but not degrading significantly when varied between 0.1 and 1.

4.2. False-positive & false-negative bounds

Taking the above into account, we derived a theorem [16] about various false-positive and false-negative bounds, i.e., bounds for erroneously detecting or missing a concept drift within a sliding window W , respectively. Theorem 4.1 below thus constitutes the main theoretical result of OPTWIN, with its proof detailed in Appendix A. Let μ_{hist} and μ_{new} denote the average values of W_{hist} and W_{new} , respectively, and let σ_{hist} and σ_{new} represent their corresponding standard deviations.

Theorem 4.1.

- **False-Positive Bound.** At every step, if μ_W and σ_W^2 remain constant within W , OPTWIN will flag a concept drift at this step with a confidence of at most $1 - \delta$.
- **False-Negative Bound (for mean drift with large enough W).** For any partitioning of W into two sub-windows W_{hist} W_{new} , with $|W| \geq w_{proof}$ and W_{new} containing the most recent elements, if $\mu_{hist} - \mu_{new} > \rho \sigma_{hist}$, then, with confidence δ , OPTWIN flags a concept drift in at most $|W| - v_{split}$ steps.
- **False-Negative Bound (for mean drift with small W).** For any partitioning of W into two sub-windows W_{hist} W_{new} , with $w_{min} \leq |W| < w_{proof}$ and W_{new} containing the most recent elements, if $\mu_{hist} - \mu_{new} > \rho_{temp} \sigma_{hist}$, then, with confidence δ , OPTWIN flags a concept drift in at most $\frac{|W|}{2}$ steps.
- **False-Negative Bound (for standard-deviation drift with any W).** For any partitioning of W into two sub-windows W_{hist} W_{new} , with $|W| \geq w_{min}$ and W_{new} containing the most recent elements, if $\frac{\sigma_{new}^2}{\sigma_{hist}^2} > f_ppf(\delta', v|W|-1, (1-v)|W|-1)$, then, with confidence δ , OPTWIN flags a concept drift in at most v_{split} steps.

5. Convergence analysis of DASGD

In this section, we present the setup under which we prove an upper bound for the convergence rate of *decentralized and asynchronous SGD* (DASGD). Our setting is based on the one presented in [35], which provides tight convergence rates for ASGD under various centralized settings. We in particular generalize their analysis of ASGD under a homogeneous setting with a fixed stepsize (their Theorem 6), and extend this also to a decentralized setting which no longer relies on a centralized parameter server.

5.1. Optimization objective

Following [35], we consider the *optimization objective* shown below:

$$\min_{x \in \mathbb{R}^d} \left[f(x) := \frac{1}{n} \sum_{i=1}^n [f_i(x) = \mathbb{E}_{\xi \sim D} F_i(x, \xi)] \right] \quad (2)$$

Here, F_i represents a local loss function which is accessed by node i with parameters (i.e., “model weights”) x on data samples $\xi \sim D$, considering $i \in \{1, \dots, n\}$ (in the following abbreviated as $i \in [n]$). As in [35], each $f_i(x)$ with $f_i : \mathbb{R}^d \rightarrow \mathbb{R}$ is assumed to be a stochastic function, i.e., $f_i(x) = \mathbb{E}_{\xi \sim D_i} F_i(x, \xi)$, and is accessed only via its local gradients $\nabla F_i(x, \xi)$.

We remark that this is a very generic setting, which captures any data distribution D and applies to any (smooth, but possibly non-convex) objective function. In case the optimization problem is, for example, deterministic, we may set $f_i(x) = F_i(x, \xi)$, $\forall \xi$; if, on the other hand, D is a uniform distribution with local samples $\{\xi_1^1, \dots, \xi_i^{m_i}\}$, we have $f_i(x) = \frac{1}{m_i} \sum_{t=1}^{m_i} F_i(x, \xi_t^i)$. However, as opposed to [35], we assume all samples ξ to come from the same global distribution D rather than allowing different local distributions D_i (which, together with fixed stepsizes, matches the setting of Theorem 6 in [35]). Consequently, we also only look at homogeneous functions $f_i(x)$, as stated in [Assumption 2](#) in Section 5.2.

Throughout this paper, we refer to L_2 -norm $\|\cdot\|_2$ as our default norm for vectors and thus simplify our notation by writing $\|\cdot\|$. Moreover, with $|v|$, for a vector $v \in \mathbb{R}^d$, we denote a corresponding vector consisting of the absolute values along v 's dimensions, i.e., $|v| = \langle |v_1|, |v_2|, \dots, |v_d| \rangle$.

5.2. Assumptions

The following assumptions are considered throughout the description of our setting and the convergence analysis provided in [Appendix B](#).

Assumption 1 (Bounded Variance). *There exists a constant σ , such that $\forall i \in [n], \forall x \in \mathbb{R}^d$:*

$$\mathbb{E}_{\xi \sim D} \|\nabla F_i(x, \xi) - \nabla f_i(x)\|^2 \leq \sigma^2 \quad (3)$$

Assumption 2 (Function Homogeneity). *The functions f_i are homogeneous, thus $\forall i, j \in [n], \forall x \in \mathbb{R}^d$:*

$$\|\nabla f_i(x) - \nabla f_j(x)\| = 0 \quad (4)$$

Assumption 3 (Lipschitz Gradient). *The gradient is L -smooth and there exists a constant $L \geq 1$, such that $\forall i \in [n], \forall x, y \in \mathbb{R}^d$:*

$$\|\nabla f_i(y) - \nabla f_i(x)\| \leq L\|x - y\| \quad (5)$$

Assumption 4 (Bounded Gradient). *There exists a constant $Q \geq 0$, such that $\forall i \in [n], \forall x \in \mathbb{R}^d$:*

$$\|\nabla f_i(x)\|^2 \leq Q^2 \quad (6)$$

The above assumptions are very common in the context of SGD and have been adopted by various classical works. Specifically, [Assumptions 1](#) and [3](#) are commonly used for most SGD proofs. The only restriction we make when compared to other papers is to consider $L \geq 1$ (instead of the commonly used $L \geq 0$), which allows us to perform simplifications during our proof that guarantee the provided convergence rate without further theoretical or practical implications. For [Assumption 4](#), we adopt the same strategy as [35] by providing two different bounds, one considering this assumption and one not considering it. At last, considering [Assumption 2](#) (also explored by many works such as [38, 58]), we focus our analysis on distributed gradient computations as they are typically performed in cloud- or HPC-based settings [59], and in which all worker nodes (using CPUs or GPUs) process data from the same distribution D .

5.3. Network topology & communication protocol

Our convergence proof for DASGD is flexible enough to allow any network topology or communication protocol between the worker nodes (e.g., fully connected, ring, and mesh topologies [30]) as long as the following characteristics (commonly present in HPC settings) are respected.

- *connected graph*: there must exist a communication path between any two nodes in the network;
- *no lost messages*: a message (encoding gradients) that is sent by a worker node shall eventually be received by all other nodes in the network;
- *no repeated messages*: each message will be received at most once by each worker node.

We remark that neither our DASGD algorithm ([Algorithm 3](#)) nor its convergence analysis depend on the order of the sent messages to be preserved among the worker nodes. We see this as a strong feature of our approach which enhances the flexibility of the communication protocol that may be adopted. In practice, the longer it takes for two nodes to exchange their local gradients, the larger the staleness between their models will become and the worse the bounds for the convergence rate will be (cf. [Appendix A](#)).

5.4. Decentralized & asynchronous SGD algorithm

We next introduce our *DASGD algorithm*, as summarized in [Algorithm 3](#). From a practical perspective, the idea behind it is to calculate new gradients only if no updates (i.e., gradients received from other models) are available. In doing so, we reduce the dissimilarity between the asynchronous models throughout the training. Specifically, we eliminate idle times coming from synchronization barriers between nodes (common on synchronous SGD) and prevent bottlenecks on parameter servers (common on centralized SGD).

Algorithm 3 DASGD Algorithm [52].

```

1: Constructor DASGD ( $x^0, i, \text{broadcast\_frequency}$ )
2:    $t = 0$ 
3:    $x_i^t = x^0$ 
4:    $\Delta = \emptyset$ 
5:    $\text{delta\_count} = 0$ 
6:    $\text{bf} = \text{broadcast\_frequency}$ 
7: procedure TRAINMODEL
8:   while true do
9:      $\Delta_{\text{apply}} = \text{receiveGradient}()$  ▷ //NULL if there is no incoming gradient
10:    if  $\Delta_{\text{apply}} \neq \text{NULL}$  then
11:       $x_i^{t+\text{bf}} = x_i^t - \Delta_{\text{apply}}$ 
12:       $t = t + \text{bf}$ 
13:    else if  $\exists \xi_i^t$  then ▷ //compute a new gradient
14:       $g_i^t = \nabla F(x_i^t, \xi_i^t)$ 
15:       $x_i^{t+1} = x_i^t - \eta g_i^t$ 
16:       $\Delta = \Delta + \eta g_i^t$ 
17:       $\text{delta\_count} = \text{delta\_count} + 1$ 
18:      if  $\text{delta\_count} = \text{bf}$  then
19:         $\text{sendGradient}(\Delta)$  ▷ //non-blocking
20:         $\Delta = \emptyset$ 
21:         $\text{delta\_count} = 0$ 
22:       $t = t + 1$ 

```

In [Algorithm 3](#), we show the protocol performed by all $i \in [n]$ worker nodes. First, all nodes are initialized with the same model parameters x^0 , their node id i , and a local iteration counter $t = 0$. We also define the frequency bf in which the models will broadcast their local gradients. At last, we initialize another two variables: (1) Δ , representing the sum of local gradients calculated by model x_i until they are broadcast; and (2) delta_count , a counter describing the number of gradients summed into Δ .

During training, from Lines 9 to 12, node i checks if any sum of gradients Δ_{apply} calculated by other models is available (Line 9). If so,

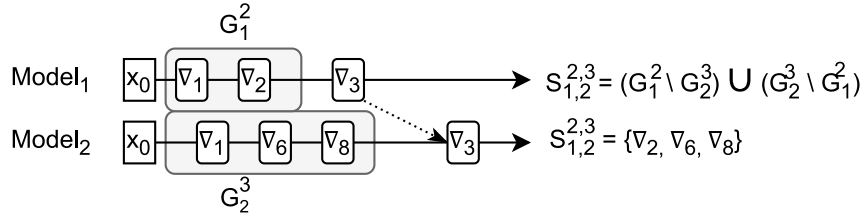


Fig. 3. Staleness $S_{1,2}^{2,3}$ at the time when Model₂ applies gradient ∇_3 calculated by Model₁. The gray areas determine the sets of gradients G_1^2 and G_2^3 used to calculate the symmetric difference.

it promptly updates itself based on it (Line 11). Then, it increments its iteration counter by bf (Line 12), as this is the number of gradients aggregated in each message received from other nodes.

If no gradients were available in Line 10, node i checks if new training samples ξ_i^t are available (Line 13). Then, it calculates a local gradient g_i^t based on its current model weights x_i^t (Line 14). After calculating g_i^t , the local model x_i^t updates itself based on it (Line 15) and sums it into Δ (Line 16). Later, it also increments the counter Δ (Line 17). Then, the algorithm checks if Δ is composed of bf gradients (Line 18). If so, Δ is broadcast (Line 19) and the variables Δ and Δ are reinitialized (Lines 20 & 21). At last, the local iteration counter t is incremented, and Algorithm 3 resumes the training on Line 8. The training is performed until $\nabla_i^t, \forall i \in [n]$ and all Δ were applied by all nodes $i \in [n]$, which assumes only a single (and final) synchronization point among all worker nodes.

5.5. Convergence rate analysis

Throughout the execution of Algorithm 3, a local model x_i will update itself using gradients calculated by a model x_j , with $i, j \in [n]$. As opposed to current works, considering our asynchronous and decentralized setting, we cannot guarantee that x_i and x_j represent the same model at different iteration points. Thus, neither $x_i^t = x_j^{t-\tau}$ nor $x_j^t = x_i^{t-\tau}, \forall \tau \in \mathbb{N}$ are assured, considering t as the model's iteration and τ a possible delay. Strictly speaking, we cannot even assume that the iteration counters t at each model are synchronized. Therefore, we rely on a different approach to calculate the difference between models. First, we represent a model x_i^t via a set of gradients G_i^t which it either received or computed itself at iteration t .

Remark 1. The set of gradients G_i^t uniquely represents the evolution of model x^0 into x_i^t when using a fixed learning rate η . Thus, we can describe x_i^t as:

$$x_i^t = x^0 - \eta \sum_{g \in G_i^t} g \quad (7)$$

Therefore, we can quantify the dissimilarity between models based on the set of gradients which they applied to themselves since they were initialized, provided that these were initialized with the same parameters x^0 and updated with the same learning rate η . More specifically, we calculate the symmetric difference between these two sets, which produces the *staleness* of gradients [60], represented here as $S_{i,j}^{t,s}$, between pairs of models i, j and iterations t, s (cf. Fig. 3). Our convergence bounds depend on the *average* and *maximum* sizes of these staleness sets, represented as S_{avg} and S_{max} , respectively.

For the following definition, let x_j^s represent model j that calculated a gradient g_j^s at iteration s , and let x_i^t represent the model i that will apply g_j^s at iteration $t+1$. We thus equally represent models x_j^s and x_i^t via the set of gradients G_j^s and G_i^t they applied at iterations s and t , respectively.

Definition 1 (Staleness). Under Assumption 4 (bounded gradients), we define *staleness* $S_{i,j}^{t,s}$ as the symmetric difference between the sets

of gradients G_i^t and G_j^s applied by models i and j at steps t and s , respectively:

$$S_{i,j}^{t,s} = (G_i^t \setminus G_j^s) \cup (G_j^s \setminus G_i^t) \quad (8)$$

(when Assumption 4 holds)

Moreover, when Assumption 4 (bounded gradients) does not hold, we also define a generalized (i.e., “looser”) version of staleness $\hat{S}_{i,j}^{t,s}$. This version is defined as the union of (1) the symmetric difference between the sets of gradients G_i^t and G_j^s (as before); and (2) the staleness $\hat{S}_{i,k}^{t,u}$, which recursively also includes gradients G_k^u calculated by another model k at step u (and which belong to G_j^s but not to G_i^t):

$$\hat{S}_{i,j}^{t,s} = (G_i^t \setminus G_j^s) \cup (G_j^s \setminus G_i^t) \cup_{g_k^u \in (G_j^s \setminus G_i^t)} \hat{S}_{i,k}^{t,u} \quad (9)$$

(when Assumption 4 does not hold)

We highlight here that, when calculating the staleness between models x_i^t and x_j^s (using G_i^t and G_j^s), each model requires only a local iteration counter, i.e., $0 \leq t \leq T_i$ and $0 \leq s \leq T_j$ (cf. Algorithm 3). To simplify further notations, we thus refer to T as the maximum step counter T_i of the model that is currently under consideration. When analyzing the staleness $S_{i,j}^{t,s}$ between pairs of models i, j , then by convention T will represent the step counter of the left-hand model (i.e., the one of model i) throughout the rest of the paper.

Definition 2 (Average & Maximum Staleness). Let g_j^s be the gradient applied to model i at step $t+1$. Then, $|S_{i,j}^{t,s}|$ and $|\hat{S}_{i,j}^{t,s}|$ denote the sizes of the *staleness* sets between model i at step t and model j at step s . Thus, we define the *maximum* and *average staleness* among all models in the same manner for S and $\hat{S}$:

$$S_{max} := \max_{0 < i \leq n, 0 \leq t \leq T} \{|S_{i,j}^{t,s}|\} \quad (10)$$

$$S_{avg} := \max_{0 < i \leq n} \left\{ \frac{1}{(T+1)} \sum_{t=0}^T |S_{i,j}^{t,s}| \right\}$$

$$\hat{S}_{max} := \max_{0 < i \leq n, 0 \leq t \leq T} \{|\hat{S}_{i,j}^{t,s}|\} \quad (11)$$

$$\hat{S}_{avg} := \max_{0 < i \leq n} \left\{ \frac{1}{(T+1)} \sum_{t=0}^T |\hat{S}_{i,j}^{t,s}| \right\}$$

The key idea behind our DASGD approach comes from the observation that gradient applications with a fixed learning rate are both *associative* and *commutative*. Therefore, we can guarantee convergence among multiple decentralized models as long as (1) the models are initialized with the same weights; (2) the models apply the same gradients (independently of their order); (3) and the same gradients are applied with the same learning rate. Moreover, by considering our definition of staleness, we can estimate the dissimilarity between models and thereby determine the expected convergence rate of the global model.

Below, we present the central results of our convergence analysis.

Theorem 5.1. Considering Assumption 1, 2, 3, 4, and a constant stepsize $\eta \leq \frac{1}{4LS_{avg}}$, Algorithm 3 reaches $\frac{1}{T+1} \sum_{t=0}^T (\|\nabla f(x^t)\|^2) \leq \epsilon$ after

$$\mathcal{O}\left(\frac{\sigma^2}{\epsilon^2}\right) + \mathcal{O}\left(\frac{QS_{avg}}{\epsilon^{\frac{3}{2}}}\right) + \mathcal{O}\left(\frac{S_{avg}}{\epsilon}\right) \quad \text{iterations.} \quad (12)$$

Moreover, without [Assumption 4](#) and using $\eta \leq \frac{1}{4L\sqrt{\hat{S}_{avg}\hat{S}_{max}}}$, [Algorithm 3](#) reaches $\frac{1}{T+1} \sum_{t=0}^T (\|\nabla f(x^t)\|^2) \leq \epsilon$ after $\mathcal{O}(\frac{\sigma^2}{\epsilon^2}) + \mathcal{O}(\frac{\sqrt{\hat{S}_{avg}\hat{S}_{max}}}{\epsilon})$ iterations. (13)

Our detailed proof of [Theorem 5.1](#) is available in [Appendix B](#) which accompanies this submission.

6. Experimental setup

We conducted four sets of experiments to assess TensAIR. Those experiments were designed to assess TensAIR's (1) ability to converge, (2) ability to scale-out to multiple CPUs and GPUs, (3) comparison with baseline models; (4) and ability to handle real-world non-IID data.

To promote usage and improve reproducibility, we developed a Python interface² that allow users to conveniently deploy TensAIR in HPC systems. All experiments described in this section are available on GitHub.²

Below, we list the different computational setups involved in the experiments detailed below.

- **Local Server:** DELL PowerEdge R840 server with 192 cores using an MPI environment which simulates multiple ranks with no shared memory.
- **HPC:** HPC facilities of the University of Luxembourg [61]. We distributed the training process over up to 8 NVIDIA Tesla V100 GPUs divided into two nodes with 4 GPUs, 28 CPUs, and 768 GB RAM each. All our experiments were performed simulating an all-connected network topology using an MPI environment, launching one rank per GPU available with no shared memory.
- **Commodity Hardware:** HP EliteBook 860 with Intel® Core™ i7-1280P Processor and 32 GB RAM.

6.1. Theoretical bounds in practice

In these experiments, we show how the theoretical bounds for DASGD introduced in [Theorem 5.1](#) traverse to practical experiments. The experiments were run on our Local Server with bf and mini batch of size equal to 1.

Inspired by [35], we assessed DASGD by performing our first two experiments in a scenario with no stochastic noise, i.e., $\sigma = 0$. In doing so, the convergence rate of the model being optimized is reduced to $\mathcal{O}(\sqrt{\hat{S}_{avg}\hat{S}_{max}} \epsilon^{-1})$, thus depending only on the *staleness* observed during the optimization. In both experiments, we fixed $n = 2$, $\epsilon = 1e^{-12}$, $\eta = 0.002$ and varied $\hat{S}_{max}$ from 0 to 100, which in particular maintains $\eta < (4L\sqrt{\hat{S}_{avg}\hat{S}_{max}})^{-1}$.

To artificially control the staleness during the optimization, we followed two strategies: (1) we reduced the gradient calculation speed of one of the models, making it x times slower; (2) we artificially increased the time it took to calculate each gradient by 0.05 s. The first technique guarantees that one of the workers outperforms the other one, thereby increasing the *maximum staleness*. The second technique guarantees that the gradient calculation takes orders of times more than the actual gradient application, which gives the slower model the chance to calculate new gradients instead of being overloaded by the application of the gradients calculated by the fastest model (which is the case in most real-world use-cases). Thus, with both techniques described above and the other parameters fixed, we can expect $\hat{S}_{max} = x$ during the optimization. Furthermore, we estimated the error of the optimized functions based on the average of the L_2 -norm of their gradients over the last 30 iterations. We optimize the same functions as in [35]:

- **quadratic function** $f(x) = \frac{1}{2} \|Ax - b\|^2$, with $x, b \in \mathbb{R}^{10}$, $i \in [1, 10]$, $b_i \sim \mathcal{N}(0, 1)$, and $A \in \mathbb{R}^{10 \times 10}$ being a random matrix with $\lambda_{min}(A) = 1$, $\lambda_{max}(A) = 2$ (cf. [Fig. 4\(a\)](#));
- **logistic-regression function** $f(x) = \frac{1}{m} \sum_{j=1}^m \log(1 + \exp(-b_j a_j^T x))$, with $a_j \sim \mathcal{N}(0, 1)^{20}$, $x \in \mathbb{R}^{20}$, $m = 100$, and b_j sampled uniformly at random from $\{-1, 1\}$ (cf. [Fig. 4\(a\)](#)).

In addition, we analyzed the impact of staleness on the convergence rate and time of a Convolutional Neural Network (CNN) for image classification [62], which we trained using the CIFAR-10 dataset [63]. In this experiment, we set $\epsilon = 1$, $\eta = 1e^{-4}$, and varied n between 1 and 25. By varying n , we ended up increasing $\hat{S}_{max}$ indirectly—close to what we expect in a real-world scenario. The results can be seen in [Section 7.1](#).

6.2. TensAIR scalability

In this experiment, we further assess how DASGD behaves in a real-world use-case. We carried out our experiments using our HPC setup.

We trained the VGG-16 model [64] using the Tiny ImageNet [65] dataset. The idea behind this experiment is to identify how DASGD behaves when optimizing a large model such as the VGG-16, with more than 40 million parameters and 150 MB of size. Considering that DASGD is decentralized and does not follow a gossip strategy, such large models are challenging as they may cause bottlenecks in the communication process. Considering this, we defined as hyper-parameters $bf = n$, $\eta = 0.001$, and used a mini-batch of size 128. The results of the VGG-16 experiments can be visualized in [Section 7.2](#).

6.3. Baseline comparison

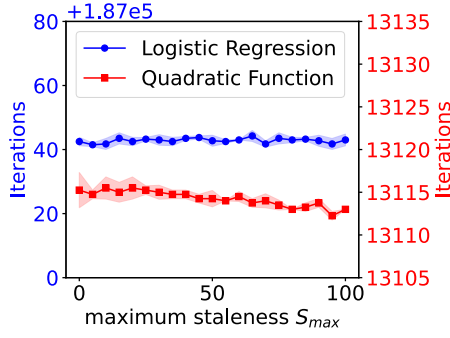
As presented in [15], in this experiment we compare the sustainable throughput of TensAIR against the existing solutions for Apache Kafka and Flink—mentioned in [Section 2](#). In practice, Apache Kafka relies on the TensorFlow I/O library (`tensorflow.io`), which does not allow any type of model distribution. On the other hand, Flink's third party library (`dl-on-flink` [13]) relies on the distributed TensorFlow API (`tensorflow.distribute`). However, we were not able to execute the provided `dl-on-flink` use-case on our HPC system. Thus, all results from the baselines are directly obtained from TensorFlow's standard and distributed APIs (the latter relying on `MirroredStrategy` from `tensorflow.distribute`), which represent upper-bounds for the achievable throughput of our baselines, thus simulating no overhead or latency due to Kafka or Flink.

To compare TensAIR with the baselines, we developed two use-cases. The first is a sparse use-case that trains a Word2Vec (W2V) model [66] on 1% of English wikipedia, corresponding to 11.7M training examples. The second is a dense use-case that trains a CNN model [62] on CIFAR-10 [63]. Both use-cases were deployed in our HPC setup using up to 4 Nvidia Tesla V100 GPUs. Different from the previous experiment, to better illustrate the importance of bf , we set $bf = 1$. For the hyper-parameters, we set mini batch of size 512 in both experiments while setting learning rate equal to 1 for the W2V use-case and 0.01 for the CIFAR-10 usecase.

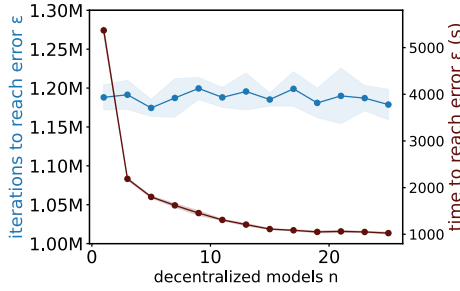
6.4. Real-world data

To better understand how TensAIR behaves under real-world concept drifts, we trained and evaluated it on real-world non-IID data. Since the other experiments already assess TensAIR's scalability and convergence, this experiment was conducted on commodity hardware without distribution, allowing us to isolate the impact of concept drift in non-IID data.

² <https://github.com/maurodl/tensair>



(a) Logistic Regression & Quadratic Function



(b) CIFAR

Fig. 4. Numbers of iterations and runtimes needed to reach an error of ϵ (each averaged over 4 runs, the shaded areas denote one standard deviation).

We trained a transformer-based model³ followed by two dense layers to predict the number of delayed flights an airport can expect from a carrier, given the scheduled flights for a given month. For this purpose, we used the “Flight Delay Data” dataset [67], which provides the monthly number of flights a carrier operated from an airport and how many of those were delayed by more than 15 min. The dataset spans from August 2013 to August 2023, covering 21 carriers and 391 airports, with a total of 171,223 records. Additional attributes present in the dataset were not used in this experiment.

The model was trained on 40% of the data using a learning rate of $\eta = 0.005$ and a mini-batch size of 32. We then evaluated prediction errors on the remaining 60% of the data using two deployment strategies: standard TensorFlow and TensAIR. For the TensAIR setup, we considered the model to have converged if the average loss over the most recent 64 mini-batches did not decrease by more than 0.3 when compared to the average loss of the previous two sets of 64 mini-batches each. For concept drift detection, we employed OPTWIN with $\rho = 0.5$. In this setting, the use of validation sets is discouraged due to temporal changes in the data distribution. Consequently, all reported error values refer to the training data.

7. Results

7.1. Results on theoretical bounds in practice

Section 7.1 illustrates the results of the “Theoretical Bounds in Practice” described in Section 6.1.

When analyzing Fig. 4, we observe no significant impact of $\hat{S}_{max}$ on the number of iterations necessary to reach ϵ . This confirms the very good convergence of DASGD in a homogeneous setting with a fixed learning rate, which is even largely invariant of $\hat{S}_{max}$ and thus

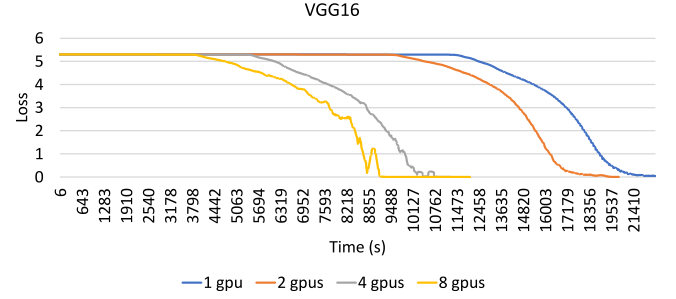
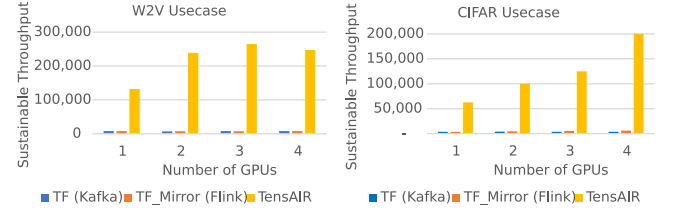


Fig. 5. VGG-16 model trained with DASGD.



(a) W2V use-case.

(b) CIFAR-10 use-case.

Fig. 6. Throughput comparison between TensAIR and TensorFlow (standard and distributed).

much better in practice than predicted by our bounds. Consequently, in Fig. 4(b), we observe a considerable decrease in the amount of time necessary to reach ϵ , which exemplifies the practical viability of DASGD.

7.2. TensAIR scalability results

Fig. 5 illustrates the results of the “TensAIR Scalability” experiment, detailed in Section 6.2.

In Fig. 5, we illustrate the convergence rate of the VGG-16 model when trained over 1, 2, 4, and 8 GPUs following Algorithm 3. We can see that, as expected, the higher the number of GPUs used during the optimization process, the lower the time it takes the model to converge. Still, differently from the experiments in Section 6.1, when training the VGG-16 model, we can verify the impact of the *staleness* during the optimization process, as given by Theorem 5.1. To reach $\epsilon = 1$ in the VGG-16 experiment, DASGD took: 125,034 iterations when training with 1 GPU; 127,475 iterations when training with 2 GPUs; 137,641 when training with 4 GPUs; and 169,801 when trained with 8 GPUs. The difference between the impact of *staleness* in the experiments in Section 6.1 and the ones presented here are mostly due to: (1) our usage of $bf = n$ in the VGG-16 experiment, which increases the model *staleness* during the optimization process; and (2) the complexity of the VGG-16 model, which made it suffer more from the “noise” generated by staled gradients. Nevertheless, as seen in Fig. 5, the speedup achieved when increasing the number of GPUs surpasses the increased number of iterations necessary to reach convergence.

7.3. Baseline comparison results

In Fig. 6, we observe that TensAIR surpasses both baselines by a great margin. This occurs because the baselines rely on the standard TensorFlow implementation. Such implementation was developed considering usual ML use-cases, in which the training data is usually transferred at most once per epoch to the distributed models. Therefore, the overhead in such transfer has a negligible impact on the total training time. However, when such transfer occurs multiple times — as it is the case for OL in which every mini-batch is transferred separately

³ <https://github.com/maurodl/TensAIR/tree/main/Examples/Flight-Delay>

— the accumulated overhead has a clear negative impact in the sustainable throughput of the system. In contrast, TensAIR was designed to mitigate such overhead by transferring data using the asynchronous iterative routing protocol. This allows TensAIR to reduce computational resources' idle times during data transfer (i.e., serialization and message passing), which optimizes its sustainable throughput.

Despite the increased throughput when compared to the baselines, in Fig. 6(a) we can observe that TensAIR's sustainable throughput did not have the same scaling factor as the other experiments presented so far. In fact, TensAIR produced lower sustainable throughput in the W2V use-case when training with 4 than with 3 GPUs. Such decrease occurred because we set $bf = 1$. In such settings, each gradient is calculated only once but transferred and applied n times (once to and by each distributed model). Therefore, in cases in which the gradient calculation does not dominate the training time by large margin — as it is the case for the W2V use-case — the additional time taken to transfer and apply the gradient directly impact the final sustainable throughput. In contrast, in the CIFAR-10 use-case (cf. Fig. 6(b)) the gradient calculation dominates the training time and the multiple gradient applications are not noticeable in the sustainable throughput. Thus, we could train with up to 200,000 images per second on CIFAR-10 with 4 GPUs, which corresponds to a sustainable throughput of 585 MB/s. To mitigate the effects seen in Fig. 6(a), one can simply define $bf = n$, as seen in Section 6.2. Still, even with $bf = 1$, we can observe that TensAIR achieves up to 78× higher sustainable throughput than the baselines when using a mini-batch size of 512.

7.4. Real-world data results

We trained the transformer model described in Section 6.4, achieving a mean absolute error (MAE) of 18.6223 on the training set after 15 epochs.

After deployment, Fig. 7 shows the monthly aggregated absolute errors of TensorFlow and TensAIR in an online setting. In this scenario, we clearly observe the impact of the COVID-19 pandemic on TensorFlow's performance: the error sharply increases at the beginning of 2020, coinciding with the global cancellation of flights.

In contrast, TensAIR quickly adapted to the effects of COVID-19 on flight delays, successfully reducing its error over time. Notably, TensAIR used 2049 samples for training and 1161 samples for prediction. This behavior reflects the relatively high number of detected concept drifts, which is expected given the long time span of the dataset and the limited number of training samples per period.

8. Discussion

DASGD for Accelerating ANN Training: Our research demonstrates that DASGD is a highly effective solution for accelerating the distributed training of ANN models. This finding underscores the significance of DASGD as a viable approach for enhancing the efficiency for the online training of ANN models.

Computation vs. Network Bottleneck: In OL scenarios with relatively few computational nodes (e.g., fewer than 16 GPUs), the research suggests that the bottleneck in real-time ANN training typically lies in the computation phase, rather than in network-related issues, especially when utilizing high-performance network technologies like InfinityBand.

Optimal GPU Utilization: The research indicates that using more than 16 GPUs in OL settings may not be beneficial for most use-cases. This limitation is attributed to the fact that the amount of data generated in real-time may not require the computational power provided by a large number of GPUs. For example, in our experiments, utilizing 4 GPUs allowed us to process image data at a rate of 585 MB/s. Remarkably,

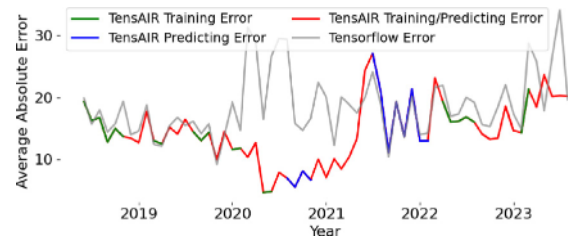


Fig. 7. Monthly aggregated errors from TensorFlow and TensAIR when predicting flight delays from a data stream.

this throughput is 11 times greater than that typically required by 4K cameras, which usually generate data at rates below 50 MB/s.

Use-Case Specific Challenges: The research recognizes that the OL setting is characterized by numerous use-case-specific challenges that cannot be comprehensively covered in a single study. These challenges often demand individual investigation and tailored solutions based on the specific problem domain.

Self-Supervised Online Training: TensAIR's primary utility lies in online training self-supervised problems, where labels can be defined automatically from the data stream. Basterrech [7] exemplifies with a Fake News detection use-case and we exemplify in [15] with a Sentiment Analysis use-case how real-time adaptation to changing data distributions can be achieved without the need for pre-defined labels. This approach has the potential to extend to other use-cases, such as audio and video data streams, where real-time embeddings can be used to adapt to data-distribution changes.

9. Conclusion

In this paper, we presented a comprehensive pipeline for training ANN models in an OL setting, achieved by optimizing TensAIR to actively adapt to concept drifts. We enhanced the DASGD algorithm by introducing the bf variable, which improves scalability in scenarios where gradient calculations are not the primary bottleneck. By theoretically proving DASGD convergence and empirically evaluating TensAIR in an HPC setting, we demonstrated its ability to achieve state-of-the-art theoretical convergence while delivering up to 78× higher sustainable throughput compared to the baseline methods. We aim to inspire researchers and developers to explore online adaptation of ANN models as a viable solution for tackling high-dimensional and complex problems, highlighting novel use-cases in this domain. Finally, we emphasize the need for further research on how different hardware setups, such as the use of Remote Direct Memory Access (RDMA) instead of message passing (MPI), may impact TensAIR's convergence.

CRedit authorship contribution statement

Mauro Dalle Lucca Tosi: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Martin Theobald:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used OpenAI GPT 3.5 and 4.0 to improve the readability and language of the manuscript. The authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Mauro Dalle Lucca Tosi reports financial support was provided by Luxembourg National Research Fund. Mauro Dalle Lucca Tosi reports a relationship with University of Luxembourg that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the Luxembourg National Research Fund under the PRIDE Programme (PRIDE17/12252781).

Appendix A. Proof of Theorem 4.1

We next prove the individual parts of Theorem 4.1 from Section 4.

Part 1. In each iteration of OPTWIN, we apply the t - and the f -tests to identify the concept drifts. Therefore, the false-positive bounds of OPTWIN are directly drawn from those tests. Thus, with $\delta' = \delta^{\frac{1}{4}}$ as the confidence for each of the four tests, we identify a false-positive drift with a confidence of at most $1 - \delta$.

Part 2: Considering an unequal-variance t -test [53] by comparing W_{hist} and W_{new} , we have:

$$t_value = \frac{\mu_{hist} - \mu_{new}}{\sqrt{\frac{\sigma_{hist}^2}{|W_{hist}|} + \frac{\sigma_{new}^2}{|W_{new}|}}} \quad (A.1)$$

The nominator on the right represents the difference observed between the means of W_{hist} and W_{new} . If this difference is large enough, we can statistically guarantee that both sets have distributions with different means. To simplify the equation, we represent this difference in terms of σ_{hist} . Therefore, we set $\mu_{hist} - \mu_{new} = \rho \sigma_{hist}$ with ρ being a user-defined robustness parameter. Thus, by applying this substitution and passing the denominator to the other side of the equation, we have:

$$\rho \sigma_{hist} = t_value \sqrt{\frac{\sigma_{hist}^2}{|W_{hist}|} + \frac{\sigma_{new}^2}{|W_{new}|}} \quad (A.2)$$

Considering ν as the splitting point of W into W_{hist} and W_{new} , we have $|W| = \nu |W_{hist}| + (1 - \nu) |W_{new}|$, and therefore:

$$\rho \sigma_{hist} = t_value \sqrt{\frac{\sigma_{hist}^2}{\nu |W|} + \frac{\sigma_{new}^2}{(1 - \nu) |W|}} \quad (A.3)$$

By our definition of a concept drift, σ_{new} and σ_{hist} must follow the same distribution (otherwise a drift occurred). Furthermore, we also consider that they follow the f -distribution, which is usually the case for standard deviations. Thus, consider the definition of the test statistic for the f -test [54] as shown below:

$$f_value = \frac{\sigma_{new}^2}{\sigma_{hist}^2} \quad (A.4)$$

We can now determine σ_{new} 's upper bound in terms of σ_{hist} and by defining f_factor as the highest value that the f_value can have while maintaining the null hypothesis of the f -test with confidence δ' .

$$\sigma_{hist}^2 f_factor \geq \sigma_{new}^2 \quad (A.5)$$

In this case, the f_factor can be calculated using the f_ppf , the PPF of the F -curve, by taking a confidence of δ' and $|W_{new}| - 1$ and $|W_{hist}| - 1$ as the degrees of freedom for the numerator and the denominator, respectively (cf. Eq. (A.4)). Thus, we obtain:

$$f_factor = f_ppf(\delta', \nu |W| - 1, (1 - \nu) |W| - 1) \quad (A.6)$$

Therefore, to later simplify Eq. (A.3), we substitute σ_{new} with its upper bound as calculated by the f -test on Eq. (A.5):

$$\rho \sigma_{hist} = t_value \sqrt{\frac{\sigma_{hist}^2}{\nu |W|} + \frac{\sigma_{hist}^2 f_factor}{(1 - \nu) |W|}} \quad (A.7)$$

Next, we can simplify the equation by removing σ_{hist} from both sides as follows:

$$\rho = t_value \sqrt{\frac{1}{\nu |W|} + \frac{f_factor}{(1 - \nu) |W|}} \quad (A.8)$$

Then, we calculate the t_value in Eq. (A.9) analogously by using the PPF of the T -curve.

$$t_value = t_ppf(\delta', d f) \quad (A.9)$$

We consider a confidence level of δ' and the degrees of freedom $d f$ for the unequal-variance t -test (cf. Eq. (A.10)).

$$d f = \frac{(\frac{\sigma_{hist}^2}{|W_{hist}|} + \frac{\sigma_{new}^2}{|W_{new}|})^2}{(\frac{\sigma_{new}^2}{|W_{hist}|})^2 + (\frac{\sigma_{new}^2}{|W_{new}|})^2} \quad (A.10)$$

By considering Eq. (A.10) in our setting with $|W_{hist}| = \nu |W|$ and $|W_{new}| = (1 - \nu) |W|$ and our upper bound for $\sigma_{new} \leq \sigma_{hist} f_factor$ (cf. Eq. (A.5)), we reach Eq. (A.11), which depicts our calculation of the degrees of freedom $d f$.

$$d f = \frac{(\frac{1}{\nu |W|} + \frac{f_ppf}{(1 - \nu) |W|})^2}{(\frac{1}{\nu |W|})^2 + (\frac{f_ppf}{(1 - \nu) |W|})^2} \quad (A.11)$$

Finally, by replacing t_value on Eq. (A.8) by the one calculated on Eq. (A.9), we reach Eq. (1) introduced in Section 4. $\square$

We remark here that Eq. (1) only depends on δ' , ρ , $|W|$ and ν . Recall that δ' is $\delta^{\frac{1}{4}}$ which is provided by the user; ρ is also given by the user; and $|W|$ increases linearly until w_{max} . We can then calculate the highest value ν based on each $|W|$ that solves Eq. (1).

Therefore, considering the split $W = W_{hist} W_{new}$ with $W_{hist} = W_{0:\nu |W|}$ and $W_{new} = W_{\nu |W|:|W|-1}$, by applying the t -test, we guarantee that if μ_{hist} and μ_{new} diverge more than $\rho \sigma_{hist}$, we identify this divergence with a confidence of δ . Moreover, considering our OL setting, it takes no more than $(1 - \nu) |W|$ iterations for W to be divided into $W_{hist} W_{new}$ after the drift occurred.

Part 3: By setting $\nu = 0.5$ in Eq. (A.8), we have:

$$\rho_{temp} = t_value \sqrt{\frac{1}{0.5 |W|} + \frac{f_factor}{0.5 |W|}} \quad (A.12)$$

With $\nu = 0.5$ and ρ_{temp} as calculated above, we can guarantee that if μ_{hist} and μ_{new} diverge more than $\rho_{temp} \sigma_{hist}$, we identify the divergence with a confidence of δ (as guaranteed by the test procedure). Similarly to Part 2 of the proof, we achieve this in at most $(1 - \nu) |W|$ iterations, in this case, $\frac{|W|}{2}$ iterations. $\square$

Part 4: This bound is directly obtained from the f -test. By simply applying the f -test to compare two sub-windows of W , we guarantee the given confidence value, in this case δ . Again, it takes no more than $(1 - \nu) |W|$ iterations for W to be divided into $W_{hist} W_{new}$ after the drift occurred. $\square$

Appendix B. Useful inequalities, remarks, and proof of Theorem 5.1

B.1. Useful inequalities & remarks

Useful Inequalities Below, we list a number of useful inequalities to which we will refer in our proof of [Theorem 5.1](#).

Lemma B.1. In analogy to [\[35\]](#), we establish the following inequality for any set of n vectors $\{a_i\}_{i=1}^n$ with $a_i \in \mathbb{R}^d$:

$$\left\| \sum_{i=1}^n a_i \right\|^2 \leq n \sum_{i=1}^n \|a_i\|^2 \quad (\text{B.1})$$

Lemma B.2. For a vector $a \in \mathbb{R}^d$ and a multiplier $m \in \mathbb{R}$, it holds that:

$$\|ma\|^2 \leq m^2 \|a\|^2 \quad (\text{B.2})$$

Lemma B.3. Considering [Assumption 3](#), for any function f , it holds that $\forall x, y \in \mathbb{R}^d$:

$$\|\nabla f(y)\| \leq \|\nabla f(x)\| + L\|x - y\| \quad (\text{B.3})$$

Lemma B.4. From the polarization identity, we have:

$$\langle a, b \rangle = \frac{\|a\|^2 + \|b\|^2 - \|a - b\|^2}{2} \quad (\text{B.4})$$

Lemma B.5. For any $a, b \in \mathbb{R}$, it holds that:

$$(a + b)^2 \leq 2a^2 + 2b^2 \quad (\text{B.5})$$

Useful Remarks

Next, we provide the following useful remarks.

Remark 2. $\Delta_{i,j}^{t,s}$ is larger or equal to the sum of any subset $A \subseteq \hat{S}_{i,j}^{t,s}$ (scaled with η), that is:

$$\eta \sum_{g \in A} |g| \leq \Delta_{i,j}^{t,s}$$

We can guarantee this because $\Delta_{i,j}^{t,s}$ is the sum of the absolute values of all gradients in $\hat{S}_{i,j}^{t,s}$ (also scaled with η). Take as an example a set $\hat{S}_{i,j}^{t,s} = \{g_\alpha, g_\beta, g_\gamma\}$. Thus, $\Delta_{i,j}^{t,s} = \eta(|g_\alpha| + |g_\beta| + |g_\gamma|)$. Evidently, $\Delta_{i,j}^{t,s} \geq \eta|g_\alpha|$. Consequently, we can also reach the following remark.

Remark 3. Considering $|x_\alpha| = x_i^t - \eta \sum_{g \in A} |g|$ and $A \subseteq \hat{S}_{i,j}^{t,s}$, we can guarantee that:

$$x_i^t - |x_\alpha| \leq \Delta_{i,j}^{t,s} \quad (\text{B.6})$$

Remark 2 follows the same idea as **Remark 3**. By definition, $\Delta_{i,j}^{t,s} = x_i^t - |x_\alpha| - |x_\beta|$, considering $|x_\beta| = x_i^t - \eta \sum_{g \in B} |g|$ and B being the complementary set of A and $\hat{S}_{i,j}^{t,s}$, that is $B = (A \cup \hat{S}_{i,j}^{t,s})$. Thus, $x_i^t - |x_\alpha| \leq \Delta_{i,j}^{t,s}$.

B.2. Proof of [Theorem 5.1](#)

Our proof for [Theorem 5.1](#) closely follows the structure of the proofs provided in Koloskova, Stich, and Jaggi in [\[35\]](#) (specifically the ones leading to Theorem 6).

First, we define $\delta_{i,j}^{t,s} \in \mathbb{R}^d$ as the difference between two models x_i^t and x_j^s , which can also be represented as the sum of the gradients in the staleness set $\hat{S}_{i,j}^{t,s}$ scaled by η , as follows:

$$\delta_{i,j}^{t,s} = \sum_{g \in \hat{S}_{i,j}^{t,s}} \eta g \quad (\text{B.7})$$

Then, we represent its upper bound $\Delta_{i,j}^{t,s} \in \mathbb{R}^d$ as the summation of the absolute values of the gradients in $\hat{S}_{i,j}^{t,s}$ (also scaled by η), expressed as follows:

$$\Delta_{i,j}^{t,s} = \sum_{g \in \hat{S}_{i,j}^{t,s}} \eta |g| \geq \delta_{i,j}^{t,s} \quad (\text{B.8})$$

Now, we define [Lemma B.6](#) which will be used to reach the convergence rates of [Eqs. \(12\) and \(13\)](#).

Lemma B.6 (Descent Lemma). Considering [Assumption 1, 2 and 3](#) with a stepsize $\eta \leq \frac{1}{2L}$, we have:

$$\begin{aligned} \mathbb{E}_{t+1} [f(x_i^{t+1})] &\leq f(x_i^t) - \frac{\eta}{2} \|\nabla f(x_i^t)\|^2 \\ &\quad + L\eta^2 \sigma^2 + \frac{\eta L^2}{2} \|\Delta_{i,j}^{t,s}\|^2 \end{aligned} \quad (\text{B.9})$$

Proof.

Following [\[35,40\]](#) and due to the L-smoothness of f , when model x_i^t updates itself with a gradient computed by model x_j^s , it then holds that:

$$\begin{aligned} \mathbb{E}_{t+1} [f(x_i^{t+1})] &= \mathbb{E}_{t+1} \left[f(x_i^t - \eta \nabla F(x_i^t + \delta_{i,j}^{t,s}, \xi_j^s)) \right] \\ &\leq f(x_i^t) - \underbrace{\eta \mathbb{E}_{t+1} [\langle \nabla f(x_i^t), \nabla F(x_i^t + \delta_{i,j}^{t,s}, \xi_j^s) \rangle]}_{T_1} \\ &\quad + \underbrace{\mathbb{E}_{t+1} \left[\frac{L\eta^2}{2} \|\nabla F(x_i^t + \delta_{i,j}^{t,s}, \xi_j^s)\|^2 \right]}_{T_2} \end{aligned}$$

Note that, due to the function homogeneity assumption ([Assumption 2](#)), we omit the indices of F and f throughout our proofs to simplify their notations.

We first transform T_1 as follows:

$$\begin{aligned} T_1 &= -\eta \mathbb{E}_{t+1} [\langle \nabla f(x_i^t), \nabla F(x_i^t + \delta_{i,j}^{t,s}, \xi_j^s) \rangle] \\ &= -\eta \langle \nabla f(x_i^t), \nabla f(x_i^t + \delta_{i,j}^{t,s}) \rangle \\ &\stackrel{(\text{B.4})}{=} -\frac{\eta}{2} \|\nabla f(x_i^t)\|^2 - \frac{\eta}{2} \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \\ &\quad + \frac{\eta}{2} \|\nabla f(x_i^t) - \nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \end{aligned}$$

Next, we transform T_2 as follows:

$$\begin{aligned} T_2 &= \mathbb{E}_{t+1} [\|\nabla F(x_i^t + \delta_{i,j}^{t,s}, \xi_j^s)\|^2] \\ &\stackrel{(3)}{\leq} \sigma^2 + \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \end{aligned}$$

Then, we combine again T_1 and T_2 :

$$\begin{aligned} \mathbb{E}_{t+1} [f(x_i^{t+1})] &\leq f(x_i^t) - \frac{\eta}{2} \|\nabla f(x_i^t)\|^2 - \frac{\eta}{2} (1 - L\eta) \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \\ &\quad + \frac{\eta}{2} \|\nabla f(x_i^t) - \nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 + \frac{L\eta^2 \sigma^2}{2} \end{aligned}$$

By exploiting the L-smoothness to estimate $\|\nabla f(x_i^t) - \nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \leq L^2 \|x_i^t - (x_i^t + \delta_{i,j}^{t,s})\|^2$, we obtain:

$$\begin{aligned} \mathbb{E}_{t+1} [f(x_i^{t+1})] &\stackrel{(5)}{\leq} f(x_i^t) - \frac{\eta}{2} \|\nabla f(x_i^t)\|^2 - \frac{\eta}{2} (1 - L\eta) \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \\ &\quad + \frac{\eta L^2}{2} \|x_i^t - (x_i^t + \delta_{i,j}^{t,s})\|^2 + L\eta^2 \sigma^2 \end{aligned}$$

Simplifying the fourth term, we get:

$$\begin{aligned} \mathbb{E}_{t+1} [f(x_i^{t+1})] &\leq f(x_i^t) - \frac{\eta}{2} \|\nabla f(x_i^t)\|^2 - \frac{\eta}{2} (1 - L\eta) \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 \\ &\quad + \frac{\eta L^2}{2} \|\delta_{i,j}^{t,s}\|^2 + L\eta^2 \sigma^2 \end{aligned}$$

By applying $\eta \leq \frac{1}{2L}$, we then obtain:

$$\leq f(x_i^t) - \frac{\eta}{2} \|\nabla f(x_i^t)\|^2 - \frac{\eta}{4} \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2 + L\eta^2 \sigma^2 + \frac{\eta L^2}{2} \|\delta_{i,j}^{t,s}\|^2$$

By discarding $-\frac{\eta}{4} \|\nabla f(x_i^t + \delta_{i,j}^{t,s})\|^2$ from the right-hand side of the inequality and considering $\Delta_{i,j}^{t,s} \geq \delta_{i,j}^{t,s}$, we reach [Lemma B.6](#). $\square$

B.3. Preliminaries for the proof of Theorem 5.1 with the convergence rate of Eq. (13)

Lemma B.7 (Estimation of the Residual). By considering Assumption 1, 2 and a constant stepsize $\eta \leq \frac{1}{4L\sqrt{\hat{S}_{avg}\hat{S}_{max}}}$, we have:

$$\frac{1}{(T+1)} \sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{7L^2(T+1)} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2] + \frac{2\eta\sigma^2}{7L}$$

Proof.

First, we unroll $\Delta_{i,j}^{t,s}$ as follows:

$$\mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] = \mathbb{E} \left[\left\| \sum_{g \in \hat{S}_{i,j}^{t,s}} \eta |g| \right\|^2 \right]$$

Let $g_k^u \in \hat{S}_{i,j}^{t,s}$ be a gradient calculated by any model k at any iteration u , then we have:

$$= \mathbb{E} \left[\left\| \sum_{g_k^u \in \hat{S}_{i,j}^{t,s}} \eta | \nabla F(x_k^u, \xi_k^u) | \right\|^2 \right]$$

Now, by considering Lemmas B.1 and B.2 as well as Assumption 1, we have:

$$\stackrel{(B.1)}{\leq} \mathbb{E} \left[\left\| \sum_{g_k^u \in \hat{S}_{i,j}^{t,s}} \eta | \nabla f(x_k^u) | \right\|^2 \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2 \quad (B.10)$$

$$\stackrel{(B.1)}{\leq} |\hat{S}_{i,j}^{t,s}| \mathbb{E} \left[\sum_{g_k^u \in \hat{S}_{i,j}^{t,s}} \eta | \nabla f(x_k^u) |^2 \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2 \quad (B.11)$$

$$\stackrel{(B.2)}{\leq} |\hat{S}_{i,j}^{t,s}| \eta^2 \mathbb{E} \left[\sum_{g_k^u \in \hat{S}_{i,j}^{t,s}} \| \nabla f(x_k^u) \|^2 \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2 \quad (B.12)$$

By considering the L-smoothness to estimate $\nabla f(x_k^u)$, we obtain:

$$\stackrel{(B.3)}{\leq} |\hat{S}_{i,j}^{t,s}| \eta^2 \mathbb{E} \left[\sum_{g_k^u \in \hat{S}_{i,j}^{t,s}} (\| \nabla f(x_i^t) \| + L \| x_i^t - x_k^u \|^2) \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2$$

Next, consider Remark 3.

$$\stackrel{(B.6)}{\leq} |\hat{S}_{i,j}^{t,s}| \eta^2 \mathbb{E} \left[\sum_{g \in \hat{S}_{i,j}^{t,s}} (\| \nabla f(x_i^t) \| + L \| \Delta_{i,j}^{t,s} \|^2) \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2$$

Let us assume $\eta \leq \frac{1}{4L\sqrt{\hat{S}_{avg}\hat{S}_{max}}}$ in the first term of the right-hand side of the inequality and $|\hat{S}_{i,j}^{t,s}| \leq \hat{S}_{max}$. We then obtain:

$$\leq \frac{1}{16L^2\hat{S}_{avg}} \mathbb{E} \left[\sum_{g \in \hat{S}_{i,j}^{t,s}} (\| \nabla f(x_i^t) \| + L \| \Delta_{i,j}^{t,s} \|^2) \right] + |\hat{S}_{i,j}^{t,s}| \eta^2 \sigma^2$$

By summing over all steps $t \in T$, we obtain:

$$\sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \stackrel{(11)}{\leq} \frac{1}{16L^2\hat{S}_{avg}} \sum_{i=0}^T \sum_{g \in \hat{S}_{i,j}^{t,s}} \mathbb{E} [\| \nabla f(x_i^t) \| + L \| \Delta_{i,j}^{t,s} \|^2] + (T+1)\hat{S}_{avg}\eta^2\sigma^2$$

By considering that the inner summation from the right side of the equation is bounded by $|\hat{S}_{i,j}^{t,s}|$ and is executed T times, we can simplify the equation by using Definition 2. This simplification upper bounds the number of iterations over $\nabla f(x_i^t)$ and $\Delta_{i,j}^{t,s}$ to $\hat{S}_{avg}$ times, thus leading us

to:

$$\stackrel{(11)}{\leq} \frac{1}{16L^2} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \| + L \| \Delta_{i,j}^{t,s} \|^2] + (T+1)\hat{S}_{avg}\eta^2\sigma^2$$

From Lemma B.5, we can further consider that $\mathbb{E} [\| \nabla f(x_i^t) \| + L \| \Delta_{i,j}^{t,s} \|^2] \leq \mathbb{E} [2\| \nabla f(x_i^t) \|^2 + 2L^2 \| \Delta_{i,j}^{t,s} \|^2]$, thus:

$$\sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{16L^2} \sum_{i=0}^T \mathbb{E} [2\| \nabla f(x_i^t) \|^2 + 2L^2 \| \Delta_{i,j}^{t,s} \|^2] + (T+1)\hat{S}_{avg}\eta^2\sigma^2$$

Then, by taking all terms based on $\| \Delta_{i,j}^{t,s} \|^2$ to the left side of the inequality:

$$(1 - \frac{2}{16}) \sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{16L^2} \sum_{i=0}^T \mathbb{E} [2\| \nabla f(x_i^t) \|^2] + (T+1)\hat{S}_{avg}\eta^2\sigma^2$$

Later simplifying the inequality, we reach:

$$\frac{7}{8} \sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{8L^2} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2] + (T+1)\hat{S}_{avg}\eta^2\sigma^2$$

Then, by multiplying the inequality by 8/7, we have:

$$\sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{7L^2} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2] + \frac{8(T+1)}{7} \hat{S}_{avg}\eta^2\sigma^2$$

At last, by assuming $\eta \leq \frac{1}{4L\sqrt{\hat{S}_{max}\hat{S}_{avg}}}$, and $\hat{S}_{max} \geq \hat{S}_{avg}$ such that $\sqrt{\hat{S}_{max}\hat{S}_{avg}} \geq \hat{S}_{avg}$. Then, it holds that:

$$\sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2] \leq \frac{1}{7L^2} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2] + \frac{2(T+1)}{7} L\eta\sigma^2$$

After dividing by $(T+1)$, we reach the statement of the lemma. $\square$

B.4. Proof of Theorem 5.1 with the convergence rate of Eq. (13)

We are now ready to give the proof of Theorem 5.1 with the convergence rate of Eq. (13). We start by passing $f(x_i^{t+1})$ to the right-hand side of the inequality of Lemma B.6 and $f(x_i^t)$ to the left-hand side. Then, for every model $i \in [n]$, it holds that:

$$\frac{\eta}{2} \| \nabla f(x_i^t) \|^2 \leq f(x_i^t) - \mathbb{E}_{t+1} [f(x_i^{t+1})] + L\eta^2\sigma^2 + \frac{\eta L^2}{2} \| \Delta_{i,j}^{t,s} \|^2$$

Then, we average over all $t \in T$ and divide by η . In the following, let f^* denote the value of our objective function at a local minimum ϵ .

$$\frac{1}{T+1} \sum_{i=0}^T \frac{1}{2} \mathbb{E} [\| \nabla f(x_i^t) \|^2] \leq \frac{1}{\eta(T+1)} (f(x_i^0) - f^*) + L\eta\sigma^2 + \frac{1}{(T+1)} \frac{L^2}{2} \sum_{i=0}^T \mathbb{E} [\| \Delta_{i,j}^{t,s} \|^2]$$

We next apply Lemma B.7 to the last term in order to obtain:

$$\frac{1}{T+1} \sum_{i=0}^T \frac{1}{2} \mathbb{E} [\| \nabla f(x_i^t) \|^2] \leq \frac{1}{\eta(T+1)} (f(x_i^0) - f^*) + L\eta\sigma^2 + \frac{L\eta\sigma^2}{7} + \frac{1}{14(T+1)} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2]$$

By setting $f(x_i^0) - f^* =: r_0$ and considering that $\frac{L\eta\sigma^2}{7} < L\eta\sigma^2$, we get:

$$\frac{1}{T+1} \sum_{i=0}^T \frac{1}{2} \mathbb{E} [\| \nabla f(x_i^t) \|^2] \leq \frac{r_0}{\eta(T+1)} + 2L\eta\sigma^2 + \frac{1}{14(T+1)} \sum_{i=0}^T \mathbb{E} [\| \nabla f(x_i^t) \|^2]$$

We next pass $\mathbb{E} [\|\nabla f(x'_i)\|^2]$ to the left-hand side of the inequality.

$$\frac{1}{T+1} \sum_{i=0}^T \left(\frac{1}{2} - \frac{1}{14}\right) \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{r_0}{\eta(T+1)} + 2L\eta\sigma^2$$

$$\frac{1}{T+1} \sum_{i=0}^T \frac{3}{7} \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{r_0}{\eta(T+1)} + 2L\eta\sigma^2$$

We then multiply the inequality by $\frac{7}{3}$ as follows:

$$\frac{1}{T+1} \sum_{i=0}^T \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{7r_0}{3\eta(T+1)} + \frac{14}{3}L\eta\sigma^2$$

Using $\eta \leq \frac{1}{4L\sqrt{\hat{S}_{avg}\hat{S}_{max}}}$ together with Lemma 17 from [32].

$$c\psi_T \leq 2\left(\frac{14L\sigma^2r_0}{3(T+1)}\right)^{\frac{1}{2}} + \frac{4Lr_0\sqrt{\hat{S}_{avg}\hat{S}_{max}}}{T+1}$$

This finally yields the bound of Theorem 5.1 with the convergence rate of Eq. (13).

$$\mathcal{O}\left(\frac{\sigma}{\sqrt{T}}\right) + \mathcal{O}\left(\frac{\sqrt{\hat{S}_{avg}\hat{S}_{max}}}{T}\right) \quad \square$$

B.5. Preliminaries for the proof of Theorem 5.1 with the convergence rate of Eq. (12)

Lemma B.8 (Estimation of the Residual — Bounded Gradients). *Considering Assumption 1, 2, 3 and 4 with a constant stepsize $\eta \leq \frac{1}{4LS_{avg}}$, we have:*

$$\frac{1}{T+1} \sum_{i=0}^T \mathbb{E} [\|A_{i,j}^{t,s}\|^2] \leq S_{avg}^2\eta^2Q^2 + S_{avg}\eta^2\sigma^2$$

Proof.

From Eq. (B.12), when using the tighter representation of staleness $|S_{i,j}^{t,s}|$, we obtain:

$$\leq |S_{i,j}^{t,s}|\eta^2 \mathbb{E} \left[\sum_{g_k^u \in S_{i,j}^{t,s}} \|\nabla f(x_k^u)\|^2 \right] + |S_{i,j}^{t,s}|\eta^2\sigma^2$$

By considering Assumption 4, in which $\|\nabla f(x_k^u)\|^2 \leq Q^2$, we have:

$$\mathbb{E} [\|A_{i,j}^{t,s}\|^2] \stackrel{(6)}{\leq} |S_{i,j}^{t,s}|^2\eta^2Q^2 + |S_{i,j}^{t,s}|\eta^2\sigma^2$$

Then, by averaging over all steps $t \in T$, we already obtain the statement of the lemma. $\square$

B.6. Proof of Theorem 5.1 with the convergence rate of Eq. (12)

Finally, we give the proof of Theorem 5.1 with the convergence rate of Eq. (12). We start by passing $f(x_i^{t+1})$ to the right-hand side of the inequality of Lemma B.6 and $f(x'_i)$ to the left-hand side, respectively.

$$\frac{\eta}{2} \|\nabla f(x'_i)\|^2 \leq f(x'_i) - \mathbb{E}_{t+1} [f(x_i^{t+1})] + L\eta^2\sigma^2 + \frac{\eta L^2}{2} \|A_{i,j}^{t,s}\|^2$$

Then, we average over all steps $t \in T$ and divide by η .

$$\frac{1}{T+1} \sum_{i=0}^T \frac{1}{2} \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{1}{\eta(T+1)} (f(x_0^0) - f^*) + L\eta\sigma^2$$

$$+ \frac{1}{T+1} \frac{L^2}{2} \sum_{i=0}^T \mathbb{E} [\|A_{i,j}^{t,s}\|^2]$$

We next apply Lemma 3 to the last term.

$$\frac{1}{T+1} \sum_{i=0}^T \frac{1}{2} \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{1}{\eta(T+1)} (f(x_0^0) - f^*) + L\eta\sigma^2$$

$$+ \frac{L^2 S_{avg}^2 \eta^2 Q^2}{2} + \frac{L^2 S_{avg} \eta^2 \sigma^2}{2}$$

Let again $f(x_i^0) - f^* =: r_0$. We simplify the inequality with the following conditions:

- we multiply the inequality by 2;
- let $\eta \leq \frac{1}{4LS_{avg}}$ on the last term of the inequality;
- we consider that $\frac{2L^2 S_{avg}^2 \eta^2 \sigma^2}{2} \leq \frac{L\eta\sigma^2}{4} \leq L\eta\sigma^2$.

We therefore obtain:

$$\frac{1}{T+1} \sum_{i=0}^T \mathbb{E} [\|\nabla f(x'_i)\|^2] \leq \frac{2r_0}{\eta(T+1)} + 3L\eta\sigma^2 + L^2 S_{avg}^2 \eta^2 Q^2$$

Let us assume $\eta \leq \frac{1}{4LS_{avg}}$, then together with Lemma 17 from [32], we obtain:

$$\psi_T \leq 2\left(\frac{3LS_{avg}^2r_0}{T+1}\right)^{\frac{1}{2}} + 2(L^2 S_{avg}^2 Q^2)^{\frac{1}{3}} \left(\frac{r_0}{T+1}\right)^{\frac{2}{3}} + \frac{4Lr_0 S_{avg}}{T+1}$$

This finally yields the bound of Theorem 5.1 with the convergence rate of Eq. (12).

$$\mathcal{O}\left(\frac{\sigma}{\sqrt{T}}\right) + \mathcal{O}\left(\frac{Q S_{avg}}{T^{\frac{2}{3}}}\right) + \mathcal{O}\left(\frac{S_{avg}}{T}\right) \quad \square$$

Data availability

We provide a link in the manuscript for accessing the code and experiments, all of which are available on GitHub.

References

- [1] S.C. Hoi, D. Sahoo, J. Lu, P. Zhao, Online learning: A comprehensive survey, *Neurocomputing* 459 (2021) 249–289.
- [2] R.H. Hariri, E.M. Fredericks, K.M. Bowers, Uncertainty in big data analytics: survey, opportunities, and challenges, *J. Big Data* 6 (1) (2019) 1–16.
- [3] N.C. Thompson, S. Ge, G.F. Manso, The importance of (exponentially more) computing power, 2022, arXiv preprint arXiv:2206.14007.
- [4] T. Paladini, M. Bernasconi de Luca, M. Carminati, M. Polino, F. Trovò, S. Zanero, Advancing fraud detection systems through online learning, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2023, pp. 275–292.
- [5] X. Chen, Y. Liu, G. Hong, An online learning approach to dynamic pricing and capacity sizing in service systems, *Oper. Res.* (2023).
- [6] T. Tantisriprecha, N. Soonthomphisaj, Stock market movement prediction using LDA-online learning model, in: *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing*, SNPD, IEEE, 2018, pp. 135–139.
- [7] S. Basterrech, A. Kasprzak, J. Platoš, M. Woźniak, A continual learning system with self domain shift adaptation for fake news detection, in: *2023 IEEE 10th International Conference on Data Science and Advanced Analytics, DSAA, IEEE*, 2023, pp. 1–10.
- [8] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, <http://www.deeplearningbook.org>.
- [9] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, *IEEE Trans. Knowl. Data Eng.* 31 (12) (2018) 2346–2363.
- [10] G.I. Webb, Naive Bayes, in: C. Sammut, G.I. Webb (Eds.), *Encyclopedia of Machine Learning*, Springer US, Boston, MA, 2010, pp. 713–714.
- [11] L. Breiman, Random forests, *Mach. Learn.* 45 (2001) 5–32.
- [12] M.M. Najafabadi, F. Villanustre, T.M. Khoshgoftaar, N. Seliya, R. Wald, E. Muharemagi, Deep learning applications and challenges in big data analytics, *J. Big Data* 2 (1) (2015) 1–21.
- [13] Deep learning on flink, 2022, Github, URL <https://github.com/flink-extended/dl-on-flink>. (Accessed: 05 August 2022).
- [14] Robust machine learning on streaming data using kafka and Tensorflow-IO, 2022, Tensorflow I/O tutorials, URL <https://www.tensorflow.org/io/tutorials/kafka>. (Accessed: 05 August 2022).
- [15] M.D.L. Tosi, V.E. Venugopal, M. Theobald, Tensair: Real-time training of neural networks from data-streams, in: *Proceedings of the 2024 8th International Conference on Machine Learning and Soft Computing*, 2024, pp. 73–82.
- [16] M.D.L. Tosi, M. Theobald, OPTWIN: Drift identification with optimal sub-windows, in: *2024 IEEE 40th International Conference on Data Engineering Workshops (ICDEW 2024)*, 2024, pp. 331–337, <http://dx.doi.org/10.1109/ICDEW61823.2024.00049>.

- [17] J. Kreps, N. Narkhede, J. Rao, et al., Kafka: A distributed messaging system for log processing, in: *Proceedings of the NetDB*, vol. 11, 2011, pp. 1–7.
- [18] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, K. Tzoumas, Apache Flink: Stream and batch processing in a single engine, *Bull. the IEEE Comput. Soc. Tech. Comm. Data Eng.* 36 (4) (2015).
- [19] V.E. Venugopal, M. Theobald, S. Chaychi, A. Tawakuli, AIR: A light-weight yet high-performance dataflow engine based on asynchronous iterative routing, in: *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing, SBAC-PAD*, 2020, pp. 51–58.
- [20] V.E. Venugopal, M. Theobald, D. Tasseti, S. Chaychi, A. Tawakuli, Targeting a light-weight and multi-channel approach for distributed stream processing, *J. Parallel Distrib. Comput.* (2022).
- [21] S. Zhang, W. Tan, Y. Li, A survey of online sequential extreme learning machine, in: *2018 5th International Conference on Control, Decision and Information Technologies, CoDIT*, IEEE, 2018, pp. 45–50.
- [22] J. Wang, S. Lu, S.-H. Wang, Y.-D. Zhang, A review on extreme learning machine, *Multimedia Tools Appl.* 81 (29) (2022) 41611–41660.
- [23] Q.H. Nguyen, H.-B. Ly, L.S. Ho, N. Al-Ansari, H.V. Le, V.Q. Tran, I. Prakash, B.T. Pham, Influence of data splitting on performance of machine learning models in prediction of shear strength of soil, *Math. Probl. Eng.* 2021 (2021) 1–15.
- [24] R. Hadsell, D. Rao, A.A. Rusu, R. Pascanu, Embracing change: Continual learning in deep neural networks, *Trends Cogn. Sci.* 24 (12) (2020) 1028–1040.
- [25] Y. LeCun, L. Bottou, G.B. Orr, K.-R. Müller, Efficient backprop, in: *Neural Networks: Tricks of the Trade*, 2002, pp. 9–50.
- [26] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
- [27] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *J. Mach. Learn. Res.* 12 (7) (2011).
- [28] Y. Cheng, D. Wang, P. Zhou, T. Zhang, Model compression and acceleration for deep neural networks: The principles, progress, and challenges, *IEEE Signal Process. Mag.* 35 (1) (2018) 126–136.
- [29] S. Ouyang, D. Dong, Y. Xu, L. Xiao, Communication optimization strategies for distributed deep neural network training: A survey, *J. Parallel Distrib. Comput.* 149 (2021) 52–65.
- [30] R. Mayer, H.-A. Jacobsen, Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools, *ACM Comput. Surv.* 53 (1) (2020) 1–37.
- [31] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, J. Liu, Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [32] A. Koloskova, N. Loizou, S. Boreiri, M. Jaggi, S. Stich, A unified theory of decentralized SGD with changing topology and local updates, in: *International Conference on Machine Learning*, 2020, pp. 5381–5393.
- [33] T. Ben-Nun, T. Hoefler, Demystifying parallel and distributed deep learning: An in-depth concurrency analysis, *ACM Comput. Surv.* 52 (4) (2019) 1–43.
- [34] B. Recht, C. Re, S. Wright, F. Niu, Hogwild!: A lock-free approach to parallelizing stochastic gradient descent, *Adv. Neural Inf. Process. Syst.* 24 (2011).
- [35] A. Koloskova, S.U. Stich, M. Jaggi, Sharper convergence guarantees for asynchronous SGD for distributed and federated learning, in: A.H. Oh, A. Agarwal, D. Belgrave, K. Cho (Eds.), *Advances in Neural Information Processing Systems*, 2022.
- [36] K. Mishchenko, F. Bach, M. Even, B.E. Woodworth, Asynchronous SGD beats minibatch SGD under arbitrary delays, *Adv. Neural Inf. Process. Syst.* 35 (2022) 420–433.
- [37] A. Cohen, A. Daniely, Y. Drori, T. Koren, M. Schain, Asynchronous stochastic optimization robust to arbitrary delays, *Adv. Neural Inf. Process. Syst.* 34 (2021) 9024–9035.
- [38] Y. Arjevani, O. Shamir, N. Srebro, A tight convergence analysis for stochastic gradient descent with delayed updates, in: *Algorithmic Learning Theory*, 2020, pp. 111–132.
- [39] C. Xie, S. Koyejo, I. Gupta, Asynchronous federated optimization, 2019, arXiv preprint arXiv:1903.03934.
- [40] J. Nguyen, K. Malik, H. Zhan, A. Yousefpour, M. Rabbat, M. Malek, D. Huba, Federated learning with buffered asynchronous aggregation, in: *International Conference on Artificial Intelligence and Statistics*, 2022, pp. 3581–3607.
- [41] M.T. Toghani, C.A. Uribe, Unbounded gradients in federated learning with buffered asynchronous aggregation, in: *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, 2022, pp. 1–8.
- [42] X. Wu, C. Liu, S. Magnusson, M. Johansson, Delay-agnostic asynchronous distributed optimization, in: *2023 62nd IEEE Conference on Decision and Control, CDC*, IEEE, 2023, pp. 1082–1087.
- [43] M.S. Assran, M.G. Rabbat, Asynchronous gradient push, *IEEE Trans. Autom. Control* 66 (1) (2020) 168–183.
- [44] T. Wu, K. Yuan, Q. Ling, W. Yin, A.H. Sayed, Decentralized consensus optimization with asynchrony and delays, *IEEE Trans. Signal Inf. Process. over Netw.* 4 (2) (2017) 293–307.
- [45] M. Even, H. Hendrikx, L. Massoulié, Asynchronous speedup in decentralized optimization, *IEEE Trans. Autom. Control* (2024).
- [46] K. Srivastava, A. Nedic, Distributed asynchronous constrained stochastic optimization, *IEEE J. Sel. Top. Signal Process.* 5 (4) (2011) 772–790.
- [47] S.S. Ram, A. Nedić, V.V. Veeravalli, Asynchronous gossip algorithms for stochastic optimization, in: *Proceedings of the 48th IEEE Conference on Decision and Control (CDC) Held Jointly with 2009 28th Chinese Control Conference*, IEEE, 2009, pp. 3581–3586.
- [48] X. Lian, W. Zhang, C. Zhang, J. Liu, Asynchronous decentralized parallel stochastic gradient descent, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 3043–3052.
- [49] M. Assran, N. Loizou, N. Ballas, M. Rabbat, Stochastic gradient push for distributed deep learning, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 344–353.
- [50] J. Jiang, W. Zhang, J. Gu, W. Zhu, Asynchronous decentralized online learning, *Adv. Neural Inf. Process. Syst.* 34 (2021) 20185–20196.
- [51] M. Bornstein, T. Rabbani, E.Z. Wang, A. Bedi, F. Huang, SWIFT: Rapid decentralized federated learning via wait-free model communication, in: *The Eleventh International Conference on Learning Representations*, 2023.
- [52] M. Dalle Lucca Tosi, Online Learning Using Distributed Neural Networks (Ph.D. thesis), Unilu - University of Luxembourg [Faculty of Science, Technology and Medicine], Esch-sur-Alzette, Luxembourg, 2024.
- [53] G.D. Ruxton, The unequal variance t-test is an underused alternative to student's t-test and the Mann-Whitney U test, *Behav. Ecol.* 17 (4) (2006) 688–690.
- [54] M. Mahbobi, T.K. Tiemann, Introductory business statistics with interactive spreadsheets-1st Canadian edition, Campus Manitoba, 2016, pp. 55–63.
- [55] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: *Proceedings of the 2007 SIAM International Conference on Data Mining*, SIAM, 2007, pp. 443–448.
- [56] M. Mahbobi, T.K. Tiemann, Regression basics, *Introductory Business Statistics with Interactive Spreadsheets-1st Canadian Edition*, BCcampus, 2015.
- [57] V. Verma, A comprehensive framework for residual analysis in regression and machine learning, *J. Inf. Syst. Eng. Manag.* 10 (31s) (2025) 34–46, <http://dx.doi.org/10.52783/jisem.v10i31s.4958>.
- [58] A. Agarwal, J.C. Duchi, Distributed delayed stochastic optimization, *Adv. Neural Inf. Process. Syst.* 24 (2011).
- [59] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, et al., Large scale distributed deep networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).
- [60] M.D.L. Tosi, V.E. Venugopal, M. Theobald, Convergence-time analysis of asynchronous distributed artificial neural networks, in: *5th Joint International Conference on Data Science & Management of Data, CODS/COMAD*, 2022, pp. 314–315, <http://dx.doi.org/10.1145/3493700.3493758>.
- [61] S. Varrette, H. Cartiaux, S. Peter, E. Kieffer, T. Valette, A. Olloh, Management of an academic HPC & research computing facility: The ULHPC experience 2.0, in: *Proc. of the 6th ACM High Performance Computing and Cluster Technologies Conf., HPCCT 2022*, Association for Computing Machinery (ACM), Fuzhou, China, 2022.
- [62] Convolutional neural network (CNN): Tensorflow core, 2022, URL <https://www.tensorflow.org/tutorials/images/cnn>. (Accessed on 10 May 2022).
- [63] A. Krizhevsky, G. Hinton, et al., Learning multiple layers of features from tiny images, Tech. rep, University of Toronto, Department of Computer Science, 2009.
- [64] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: Y. Bengio, Y. LeCun (Eds.), *3rd International Conference on Learning Representations, ICLR 2015*, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015.
- [65] Y. Le, X. Yang, Tiny imagenet visual recognition challenge, *CS 231N* 7 (7) (2015) 3.
- [66] TensorFlow, 2022, Tensorflow tutorials, URL <https://www.tensorflow.org/tutorials/text/word2vec>. (Accessed: 25 May 2022).
- [67] S.H. Eedala, Flight delay data — kaggle.com, 2023, Kaggle dataset, URL <https://www.kaggle.com/datasets/sriharshaedala/airline-delay>. (Accessed 30 June 2025).